Johannes Gutenberg University Mainz

Department 08 – Physics, Mathematics and Computer Science

Institute of Geosciences

Data-Driven Stokes Flow Prediction in Multi-Crystal Sedimentation:

A Systematic Comparison of Spatial and Spectral Neural Surrogate Architectures

Master's Thesis

submitted in partial fulfillment of the requirements

for the degree of

Master of Science (M.Sc.)

in

Computational Sciences

submitted by

Paul Baselt

Matriculation Number: 2706608

born in Berlin

First Supervisor:
Univ.-Prof. Dr. Boris J. P. Kaus
Institute of Geosciences
Johannes Gutenberg-Universität Mainz

Second Supervisor:
Prof. Dr. Michael Wand
Institute of Computer Science
Johannes Gutenberg-Universität Mainz

Mainz, March 7, 2026

Eidesstattliche Erklärung

Hiermit erkläre ich, dass ich die vorliegende Masterarbeit selbständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet habe. Alle Stellen, die wörtlich oder sinngemäß aus veröffentlichten oder unveröffentlichten Arbeiten entnommen sind, habe ich als solche kenntlich gemacht.

Die Arbeit ist in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegt worden.

Ort, Datum	Unterschrift

Zusammenfassung

Kristallsedimentation in viskosen magmatischen Fluiden ist ein fundamentaler geodynamischer Prozess, der die chemische Differentiation und innere Struktur von Magmakammern steuert. Die numerisch exakte Simulation von Mehrkristall-Sedimentation erfordert die gleichzeitige Auflösung mikroskopischer Grenzschichten um einzelne Kristalle und weitreichender hydrodynamischer Wechselwirkungen über das gesamte Strömungsgebiet. Dieser ausgeprägt multiskalige Charakter, kombiniert mit dem nahezu quadratischen Skalierungsverhalten der Interaktionskosten mit der Partikelzahl, macht systematische Parameterstudien mit klassischen Solvern rechnerisch kaum durchführbar.

Diese Masterarbeit untersucht, ob datengetriebene Surrogatmodelle Hochpräzisions-Stokes-Solver für die Strömungsfeldvorhersage in Mehrkristall-Sedimentationsszenarien ersetzen können. Zwei neuronale Netzarchitekturen werden auf identischen Daten und Evaluationsmetriken systematisch verglichen: das U-Net, ein Encoder-Decoder-Modell mit Skip-Connections zur Extraktion räumlich hierarchischer Merkmale, und der Fourier Neural Operator (FNO), der im Spektralbereich operiert und pro Schicht ein globales rezeptives Feld besitzt. Beide Modelle werden darauf trainiert, die Stromfunktion ψ aus einer fünfkanaligen geometrischen Eingabekodierung vorherzusagen, bestehend aus einer Kristallmaske, einem Signed-Distance-Proxy, dem Abstand zur nächsten Kristallgrenze sowie normierten Raumkoordinaten. Die Vorhersage von ψ statt direkter Geschwindigkeitskomponenten erzwingt Inkompressibilität per Konstruktion und eliminiert so eine häufige Quelle systematischer Verletzungen physikalischer Nebenbedingungen in rein datengetriebenen Strömungssurrogaten.

Trainingsdaten werden mit dem geodynamischen Finite-Elemente-Code LaMEM generiert und liefern stationäre inkompressible Stokes-Lösungen für Konfigurationen mit 1 bis $N_{\max}$ zufällig platzierten starren kreisförmigen Kristallen auf einem festen 256×256-Gitter.

Genaue Zahlen muss ich noch einpflegen sobald ich die Versuche gemacht habe.

Aus ψ abgeleitete Geschwindigkeitsfelder erweisen sich konsistent als schwieriger zu reproduzieren als die Stromfunktion selbst, mit relativen Fehlern, die rund eine Größenordnung größer sind und die Hochfrequenzverstärkung numerischer Differentiation widerspiegeln. Fehlerdiagnostik zeigt zudem, dass die Vorhersageschwierigkeit an Kristallrändern und Domänengrenzen erhöht ist.

Der kontrollierte Vergleich der U-Net- und FNO-Architekturen auf diesem Benchmark

quantifiziert den Zielkonflikt zwischen räumlich lokaler Multiskalenverarbeitung und globaler Spektraldarstellung für die Surrogatmodellierung elliptischer Stokes-Strömungen mit mehreren Einschlüssen. Die Ergebnisse liefern praktische Richtlinien für Eingabekodierung, Ausgaberepräsentation und Architekturwahl in geophysikalischen Sedimentationsanwendungen.

Abstract

Crystal sedimentation in viscous magmatic fluids is a fundamental geodynamic process that drives chemical differentiation and controls the internal structure of magma reservoirs. Accurate simulation of multi-crystal settling requires the simultaneous resolution of microscale boundary layers around individual crystals and long-range hydrodynamic interactions spanning the full computational domain. This multi-scale character, combined with the near-quadratic scaling of interaction costs with particle number, renders systematic parameter studies with classical solvers computationally prohibitive.

This thesis investigates whether data-driven surrogate models can effectively replace high-fidelity Stokes solvers for predicting flow fields in multi-crystal sedimentation scenarios. Two neural network architectures are systematically compared on identical data and evaluation metrics: the U-Net, an encoder–decoder model with skip connections that extracts spatially hierarchical features, and the Fourier Neural Operator (FNO), which operates in the spectral domain and provides a global receptive field per layer. Both models are trained to predict the stream function ψ from a five-channel geometric input encoding consisting of a crystal mask, a signed distance proxy field, the distance to the nearest crystal boundary, and normalized spatial coordinates. Predicting ψ rather than velocity components directly enforces incompressibility by construction, eliminating a common source of systematic constraint violations in purely data-driven velocity surrogates.

Training data are generated using the LaMEM geodynamic finite-element code, providing steady-state incompressible Stokes solutions for configurations of 1 to $N_{\max}$ randomly placed rigid circular crystals on a fixed 256×256 grid.

Siehe Anmerkung in der deutschen Zusammenfassung bezüglich genauer Zahlen, die ich noch einpflegen muss sobald ich die Versuche gemacht habe.

Gradient-derived velocity quantities prove consistently harder to reproduce than ψ itself, with relative errors roughly an order of magnitude larger, reflecting the amplification of residuals under numerical differentiation. Error diagnostics further show that prediction difficulty increases near crystal boundaries and domain edges.

The controlled comparison of U-Net and FNO architectures on this benchmark quantifies the trade-off between spatially local multiscale processing and global spectral representations for surrogate modeling of elliptic, multi-inclusion Stokes flow. The results provide practical guidelines for input encoding, output representation, and architecture selection in geophysical sedimentation applications.

Contents

Eidesstattliche Erklärung **III**

Zusammenfassung **V**

Abstract **VII**

I List of Figures **XIII**

II List of Tables **XV**

III Listings **XVII**

1 Introduction and Research Motivation **1**

1.1 Geoscientific Context 1
1.2 Computational Challenges 2
1.3 Machine Learning as a Surrogate Modeling Strategy 2
1.4 Research Question and Hypothesis 3
1.5 Objectives and Scope of the Study 4

2 State of the Art **7**

2.1 Deep Learning for Fluid Dynamics 7
2.2 Physics-Informed and Hybrid Approaches 8
2.3 Neural Operators and Generalization Across Geometries 9
2.4 Machine Learning for Particle- and Crystal-Laden Flows 10
2.5 Comparison of Network Paradigms for Flow-Field Surrogates 10
2.5.1 Coordinate-Based Implicit Models 11
2.5.2 Linear Reduced-Order Models 11
2.5.3 Grid-Based Convolutional Surrogates 11
2.5.4 Spectral Neural Operators 11
2.5.5 Implications for This Work 12
2.6 Position of This Thesis in the Research Landscape 12

3 Theory and Physical Background **13**

3.1 Incompressible Stokes Flow at Low Reynolds Number 13
3.2 Stream-Function Formulation in Two Dimensions 14

3.3 Vorticity and Poisson Reconstruction of the Stream Function 14
3.4 Learning Targets as Physically Motivated Inductive Biases 15
3.5 U-Net Architecture 16
3.5.1 Encoder Path 16
3.5.2 Decoder Path and Skip Connections 16
3.5.3 Multiscale Feature Extraction 16
3.6 Fourier Neural Operator 17
3.6.1 Fourier Layer 17
3.6.2 Architecture 17
3.6.3 Properties Relevant to Stokes Flow 17
3.7 Architectural Comparison: U-Net vs. FNO 18
3.8 Implications for Multi-Crystal Sedimentation Modeling 18

4 Methodology 21
4.1 Learning Target: Stream-Function Prediction 21
4.2 LaMEM Simulations for Data Generation 22
4.2.1 Domain and Discretization 22
4.2.2 Material Properties and Governing Equations 22
4.2.3 Extraction of Flow Quantities 23
4.3 Computing and Normalizing the Stream Function 23
4.3.1 Discretization and Boundary Conditions 23
4.3.2 Motivation and Procedure for Normalization 24
4.4 Input Representation: Mask, Distance Field, and Coordinates 24
4.5 Surrogate Architectures 25
4.5.1 U-Net 25
4.5.2 Fourier Neural Operator 26
4.5.3 Regularization and Practical Considerations 27
4.6 Training Procedure 27
4.6.1 Data Splits and Benchmark Design 27
4.6.2 Batching and Device Handling 27
4.6.3 Loss Function 27
4.6.4 Optimization and Checkpointing 28
4.7 Evaluation Pipeline 28
4.8 Summary 28

5 Implementation 29
5.1 Software Environment 29
5.2 Code Organization and Execution Flow 30
5.3 Model Implementations 30
5.4 Data Pipeline and Dataset Interfaces 31

5.5 Stream-Function Reconstruction and Normalization . . . 32
5.6 Training Loop and Optimization . . . 32
5.7 Evaluation Utilities . . . 33
5.8 Summary . . . 33

6 Experiments **35**
6.1 Dataset and Data Split . . . 35
6.2 Learning Task and Input Encoding . . . 35
6.3 Evaluation Metrics . . . 36
6.4 Experimental Configurations . . . 36
6.5 Training Setup and Hyperparameters . . . 37

7 Results **39**
7.1 Experiment 1: Single-Crystal Baseline . . . 39
7.1.1 FNO – Training Behavior . . . 39
7.1.2 FNO – Evaluation Metrics . . . 39
7.1.3 FNO – Qualitative Results . . . 41
7.1.4 U-Net – Training Behavior . . . 41
7.1.5 U-Net – Metrics at Epoch 50 . . . 41
7.1.6 U-Net – Qualitative Results . . . 43
7.1.7 U-Net – High-Frequency Residuals in the Velocity Field . . . 43
7.1.8 Comparison: FNO vs. U-Net . . . 43
7.2 Experiment 2: Multi-Crystal Generalization . . . 45
7.2.1 U-Net – Training Behavior . . . 45
7.2.2 U-Net – Evaluation Metrics . . . 45
7.2.3 U-Net – Qualitative Results . . . 46
7.2.4 FNO – Training Behavior . . . 46
7.2.5 FNO – Evaluation Metrics . . . 46
7.2.6 FNO – Qualitative Results . . . 48
7.2.7 Comparison: FNO vs. U-Net . . . 48
7.3 Experiment 3: Stress-Testing Generalization Limits . . . 49
7.3.1 U-Net – Training Behavior . . . 49
7.3.2 U-Net – Evaluation Metrics . . . 49
7.3.3 U-Net – Qualitative Results . . . 50
7.3.4 FNO – Training Behavior . . . 50
7.3.5 FNO – Evaluation Metrics . . . 50
7.3.6 FNO – Qualitative Results . . . 52
7.3.7 Comparison: FNO vs. U-Net . . . 52
7.4 Experiment 4: Architecture Ablation . . . 52

8 Discussion **53**

A Bibliography **55**

I List of Figures

7.1 FNO Experiment 1: validation relative L_2 error over training epochs for all four learning-rate configurations, shown separately for batch size 16 (left) and batch size 8 (right). The y-axis uses a logarithmic scale. 40

7.2 U-Net Experiment 1: validation relative L_2 error over training epochs for all four learning-rate configurations, shown separately for batch size 16 (left) and batch size 8 (right). The y-axis uses a logarithmic scale. 42

7.3 Experiment 1 evaluation metrics for all learning-rate configurations, shown separately for batch size 16 (left) and batch size 8 (right). Each group of bars shows the mean relative L_2 error on ψ and on the derived velocity field for FNO and U-Net side by side. 44

7.4 FNO Experiment 2: validation relative L_2 error and normalized training MSE over 50 training epochs. The validation error decreases steadily, in contrast to the oscillatory U-Net convergence in the same setting. 47

7.5 FNO generalization across crystal counts for Experiment 2 (left, trained on $n \leq 10$) and Experiment 3 (right, trained on $n \leq 25$). Solid lines indicate in-distribution counts; dashed lines with diamond markers indicate out-of-distribution counts. The dotted vertical line marks the boundary of the training distribution. 48

7.6 FNO Experiment 3: validation relative L_2 error and normalized training MSE over 50 training epochs (training distribution: $n \leq 25$). Convergence is steady throughout, with a slightly higher final error than in Experiment 2. 51

II List of Tables

3.1 Structural comparison of U-Net and FNO architectures. 18

6.1 Hyperparameter comparison of U-Net and FNO training configurations. . . 38

7.1 FNO evaluation metrics for Experiment 1 (single crystal, $n = 1$, 10 samples) for batch sizes 16 (upper block) and 8 (lower block). $\bar{\varepsilon}_\psi$: mean relative L_2 error on ψ; $\bar{\varepsilon}_v$: mean relative L_2 error on derived velocity; $\bar{e}_{\max}$: mean maximum absolute error on ψ; $\overline{\mathrm{MSE}}_\psi$: mean squared error on ψ. Standard deviations over the 10 samples in parentheses. Entries marked *(pending)* are not yet available. 40

7.2 U-Net metrics for Experiment 1 (single crystal, $n = 1$, 180 epochs, composite loss). $\bar{\varepsilon}_\psi^{\mathrm{eval}}$: mean relative L_2 error on ψ, held-out evaluation set (10 samples, best checkpoint); $\bar{\varepsilon}_v^{\mathrm{eval}}$: mean relative L_2 error on derived velocity; $\varepsilon_\psi^{\mathrm{val}}$: best validation relative L_2 error on ψ over all 180 epochs. . . 42

7.3 Best result per architecture and batch size in Experiment 1 (single crystal, $n = 1$, FNO: 100 epochs; U-Net: 180 epochs, composite loss), evaluated on the held-out evaluation set (10 samples). 44

7.4 U-Net evaluation metrics for Experiment 2 (trained on $n \in \{1, \ldots, 10\}$, 10 samples per stratum). OOD counts marked with †. 46

7.5 FNO evaluation metrics for Experiment 2 (trained on $n \in \{1, \ldots, 10\}$, 10 samples per stratum). OOD counts marked with †. 47

7.6 FNO vs. U-Net comparison for Experiment 2 (trained on $n \in \{1, \ldots, 10\}$). In-distribution: mean over $n = 1$–10; OOD: at $n = 25$. 48

7.7 U-Net evaluation metrics for Experiment 3 (trained on $n \in \{1, \ldots, 25\}$). OOD counts ($n > 25$) marked with †; N denotes actual sample count. . . . 50

7.8 FNO evaluation metrics for Experiment 3 (trained on $n \in \{1, \ldots, 25\}$). OOD counts ($n > 25$) marked with †; N denotes actual sample count. . . . 51

7.9 FNO vs. U-Net comparison for Experiment 3 (trained on $n \in \{1, \ldots, 25\}$). In-distribution: mean over $n = 1, 5, 10, 20, 25$; OOD: at $n = 50$. 52

III Listings

1 Introduction and Research Motivation

1.1 Geoscientific Context

Crystal sedimentation in viscous magmatic fluids is a fundamental process governing the evolution, differentiation, and internal dynamics of magma reservoirs. During cooling, crystals nucleate and grow within the melt and begin to settle due to density contrasts between the solid and liquid phases. This settling process directly influences the thermal and chemical evolution of the system and contributes to the formation of cumulate layers that are preserved as macroscopic structures in plutonic bodies. Minerals such as olivine and plagioclase are known to settle at rates that permit efficient chemical differentiation: in basaltic chambers, residence times range from days to a few years for mafic minerals and up to several centuries for plagioclase (Daniel Martin et al. 1988; D. Martin et al. 1989).

Early work by Daniel Martin et al. (1988) showed that crystal settling can occur even in vigorously convecting magma chambers, provided that the Stokes settling velocity exceeds characteristic convective flow velocities near the chamber floor. This finding called into question the long-held view that convection necessarily suppresses sedimentation, and emphasized the role of local particle–fluid interactions. A settling crystal perturbs the surrounding flow field by generating viscous boundary layers and wake structures that extend over several crystal diameters. These flow perturbations are not confined to the immediate vicinity of a single particle but can modify the trajectories and settling behavior of neighboring crystals.

Subsequent experimental and numerical investigations (D. Martin et al. 1989; Weinstein et al. 1988; Verhoeven et al. 2009; Patocka et al. 2020; Uhlmann et al. 2014; Penlou et al. 2023; Nissanka et al. 2023) revealed that such interactions may lead to asymmetric flow patterns, recirculation zones, and clustering phenomena. Weinstein et al. (1988) further showed that the spatial distribution of deposited crystals is inherently asymmetric with respect to the geometry of the convection cell, offering a physical explanation for the non-uniform mineralogical layering commonly observed in plutonic rocks. As a consequence, even small variations in crystal geometry, spatial arrangement, or relative positioning can strongly affect collective settling dynamics and the development of such layering. Taken

together, these results indicate that multi-crystal sedimentation is an inherently multiscale and interaction-dominated process, which places high demands on any numerical approach aimed at capturing the relevant physics.

1.2 Computational Challenges

Accurate numerical simulation of multi-crystal sedimentation remains computationally demanding due to the strongly multiscale nature of crystal–fluid interactions. Any physically meaningful model needs to account for:

- **Microscale crystal boundaries**, where steep velocity gradients and localized shear layers develop,
- **Long-range hydrodynamic interactions** between multiple crystals mediated by the surrounding fluid, and
- **Domain-scale flow structures** that control the large-scale transport of crystals and melt.

The computational cost of resolving hydrodynamic interactions typically scales between $\mathcal{O}(N^2)$ and $\mathcal{O}(N^3)$ with the number of particles N (Ladd 1994). Numerical methods that explicitly couple particle and fluid dynamics—such as lattice Boltzmann discrete element methods (LBM-DEM) or immersed boundary approaches—generally require each particle to be resolved by tens to hundreds of grid cells. Even in two-dimensional configurations, this leads to simulations with 10^5-10^6 fluid nodes per particle (X. Li et al. 2022; Leonardi et al. 2014). For systems containing more than ten crystals, total grid sizes can easily exceed 10^7–10^8 degrees of freedom, resulting in runtimes ranging from several hours to multiple days per simulation (X. Li et al. 2022).

Such computational demands make systematic parameter studies effectively infeasible. Investigating the influence of crystal size, viscosity contrast, particle number, or spatial arrangement would require thousands of high-fidelity simulations and thus exceed practical computational budgets. This is the main practical reason why surrogate models—capable of approximating the underlying flow fields at a fraction of the computational cost—have attracted growing interest in recent years.

1.3 Machine Learning as a Surrogate Modeling Strategy

Machine learning has increasingly been applied to accelerate predictions in fluid dynamics, with the goal of replacing expensive numerical solvers with fast, trained approximations. Two architectural paradigms are of particular relevance to this work.

First, U-Net architectures (Ronneberger et al. 2015) leverage an encoder–decoder structure with skip connections to capture multiscale spatial features. Their ability to encode complex geometric information at multiple resolutions has made them a widely used choice for structured flow-field prediction (Guo et al. 2016; Thuerey et al. 2020; Ribeiro et al. 2021; Chen et al. 2019).

Second, Fourier Neural Operators (FNOs) (Z. Li et al. 2021) learn mappings between function spaces by performing convolutions in the spectral domain via the Fast Fourier Transform. By operating in frequency space, FNOs can efficiently capture global, long-range dependencies in the solution field—a property that is particularly attractive for problems governed by elliptic operators such as the Stokes equations.

Both approaches have been shown to achieve speed-ups of several orders of magnitude over classical solvers, while maintaining acceptable accuracy across a range of flow problems. Beyond purely data-driven approaches, physics-informed neural networks (PINNs) (Maziar Raissi et al. 2017; M. Raissi et al. 2019; Jin et al. 2021) incorporate governing equations directly into the training objective, encouraging physically consistent predictions even when training data are scarce.

A broader overview of recent developments—including geometry-aware PINNs, neural operators, and scalable U-Net variants for complex flow problems—is given in Chapter 2. These advances motivate the central premise of this thesis: How well can U-Net and FNO architectures, trained on a limited number of crystal configurations, generalize to unseen sedimentation scenarios with varying particle numbers and spatial arrangements? And which architecture is better suited for this task, given their fundamentally different approaches to capturing spatial structure?

1.4 Research Question and Hypothesis

The central research question of this thesis is:

> *To what extent can U-Net and Fourier Neural Operator architectures, trained on configurations containing up to a fixed maximal number of crystals (N), generalize to unseen sedimentation scenarios with varying particle numbers and spatial arrangements—and which factors fundamentally limit this generalization capability?*

This question is of direct relevance to geoscientific applications, as natural sedimentation processes rarely involve fixed particle counts or highly regular geometries. A practically useful surrogate model must therefore generalize across a wide range of spatial configurations, interaction regimes, and crystal numbers.

We hypothesize that an ML-based surrogate model can achieve such generalization provided that:

1. the training data sufficiently sample the space of possible geometric configurations;

2. the learning task incorporates a physically meaningful target representation—such as the stream function ψ—which enforces incompressibility by construction and reduces the effective degrees of freedom of the problem;

3. the network architecture captures the inherently multiscale character of crystal–fluid interactions—whether through spatially hierarchical feature extraction (U-Net) or through global spectral representations (FNO).

The controlled comparison of U-Net and FNO architectures presented in this thesis is designed to explicitly test these hypotheses, to quantify the impact of architectural choice on predictive performance, and to identify which paradigm—local spatial convolutions or global spectral operations—is better suited for surrogate modeling of multi-crystal Stokes flow.

1.5 Objectives and Scope of the Study

Primary Objective

Quantitatively evaluate the generalization performance of U-Net and FNO based surrogate models across varying crystal numbers (1–100) and spatial arrangements on a fixed 256×256 computational grid.

Secondary Objective

Identify recurring prediction errors and characteristic failure patterns encountered when predicting previously unseen geometries, including symmetry-breaking artifacts, nonphysical divergence, and reduced accuracy near crystal boundaries.

Tertiary Objective

Derive practical guidelines for the construction of surrogate models for multiphase Stokes flow in geophysical settings, with a focus on input representations, output variables, normalization strategies, and training methodologies.

Scope Limitations

To maintain computational feasibility while addressing the core research questions, this study is restricted to:

- two-dimensional incompressible Stokes flow formulated in stream-function-vorticity form (a standard starting point for such problems that captures the essential physics of crystal–fluid interactions (Weinstein et al. 1988; Verhoeven et al. 2009)),
- rigid circular crystal geometries without collision or lubrication forces,
- steady-state flow fields defined on a fixed 256×256 grid,
- synthetic training data generated using the LaMEM code (Popov et al. 2013),
- pre-generated training, validation, and evaluation datasets stored on disk and loaded during training to ensure reproducibility and consistent benchmarking across architectures.

These assumptions allow for a focused investigation of surrogate modeling strategies while preserving the essential physical characteristics of multi-crystal sedimentation dynamics.

Structure of the Thesis

The remainder of this thesis is organized as follows. Chapter 2 reviews recent machine-learning-based surrogate models for fluid dynamics and situates the present work within the existing literature. Chapter 3 introduces the physical and mathematical foundations of Stokes flow, the stream-function formulation, and the U-Net and FNO architectures employed for structured flow-field prediction. Chapter 4 describes the computational workflow employed in this thesis, including LaMEM data generation, stream-function computation, input construction, network architecture, and training and evaluation procedures. Chapter 5 details the technical realization of the surrogate modeling framework, covering the software environment, model implementations, data pipeline, and training infrastructure. Chapter 6 presents the experimental setup, including the choice of hyperparameters, training configurations, and evaluation metrics used throughout the study. Chapter 7 reports the numerical results obtained from the U-Net and FNO surrogate models across varying crystal numbers and spatial arrangements. Chapter 8 interprets the results in the context of the research questions and hypotheses, identifies key insights and limitations, and discusses implications for future surrogate modeling efforts in geophysical fluid dynamics.

2 State of the Art

This chapter reviews recent developments in machine-learning-based surrogate models for fluid dynamics, with a particular emphasis on incompressible Stokes flow, flow–structure interaction, and neural architectures operating on structured grids. While classical numerical methods remain the reference standard in terms of physical fidelity, their computational cost often renders large parameter studies infeasible. This limitation has motivated the growing interest in data-driven surrogate models that aim to approximate solution operators at a fraction of the computational expense.

The literature surveyed in this chapter provides the conceptual and methodological foundation for the modeling choices made in this thesis. In particular, it highlights both the opportunities and the limitations of existing surrogate approaches when applied to problems involving complex, geometry-dependent flow fields.

2.1 Deep Learning for Fluid Dynamics

Deep neural networks have proven effective for approximating solutions of partial differential equations, particularly in fluid dynamics. Convolutional neural networks (CNNs) and encoder–decoder architectures such as the U-Net have shown strong results for mapping geometric input fields to structured solution fields (Ronneberger et al. 2015; Thuerey et al. 2020). Their success can be attributed to a combination of local feature extraction, multiscale representation, and translation-invariant convolutional operators.

Early studies on CNN-based surrogates for steady laminar flows showed that these models can reproduce high-fidelity Navier–Stokes or Stokes solutions with speed-ups of several orders of magnitude over traditional solvers, while maintaining acceptable accuracy (Guo et al. 2016; Ribeiro et al. 2021; Chen et al. 2019). These results confirmed that deep learning can serve as a viable surrogate modeling strategy in computational fluid dynamics.

Subsequent work extended these approaches to more complex geometries and boundary conditions. In particular, scalable U-Net variants for flow prediction in multi-obstacle environments showed that multiscale encoder–decoder architectures with skip connections are well suited for problems that require simultaneous resolution of sharp boundary layers and global flow structures (Thuerey et al. 2020; Rana et al. 2024). This combination of local and global context is directly relevant for multi-crystal sedimentation, where lo-

calized wake structures around individual crystals interact with domain-scale circulation patterns. U-Net surrogates have further been applied to low-Reynolds-number flows in microfluidic channels (Le et al. 2022), demonstrating their suitability for the creeping-flow regime. Bayesian extensions of the encoder–decoder paradigm additionally provide predictive uncertainty estimates alongside flow-field predictions (Zhu and Zabaras 2018), which may be informative when assessing surrogate reliability across varying geometric configurations. The generalization behavior of such surrogates—i.e. their ability to transfer to previously unseen configurations—has been systematically analyzed by Morimoto et al. (2022), who evaluate neural networks on multi-obstacle flows and identify representational and architectural factors limiting out-of-distribution performance.

2.2 Physics-Informed and Hybrid Approaches

Physics-Informed Neural Networks (PINNs) aim to incorporate physical knowledge directly into the learning process by augmenting the loss function with residuals of the governing partial differential equations (Maziar Raissi et al. 2017; M. Raissi et al. 2019). Rather than relying exclusively on paired input–output data, PINNs enforce approximate satisfaction of the underlying equations at collocation points within the computational domain. For incompressible flow, this typically involves penalizing the momentum equations together with the divergence-free constraint (Jin et al. 2021). PINNs have also been applied to flow problems with rigid inclusions, demonstrating their ability to predict velocity and pressure fields around circular and elliptical particles at low Reynolds numbers (Hu et al. 2023).

Hybrid extensions of this idea include geometry-aware PINNs, which embed geometric information through distance functions or latent representations to improve learning on irregular domains (Oldenburg et al. 2022). A complementary paradigm, physics-constrained deep learning, enforces conservation laws through architectural or loss-based constraints rather than full PDE residuals (Zhu, Zabaras, et al. 2019). While these approaches can enhance physical consistency, they often introduce additional challenges. In practice, PINNs and hybrid formulations tend to be sensitive to the relative weighting of loss terms, exhibit slow convergence, and incur significant computational overhead due to repeated evaluation of spatial derivatives.

These difficulties become particularly pronounced for creeping Stokes flow with multiple rigid inclusions, where sharp gradients near particle boundaries coexist with long-range hydrodynamic interactions. For this reason, the present thesis does not adopt a full physics-informed formulation. Instead, physical structure is incorporated implicitly through the choice of target representation, most notably by predicting the stream function ψ, which enforces incompressibility by construction while retaining the efficiency of a purely data-driven training procedure.

2.3 Neural Operators and Generalization Across Geometries

Neural operators seek to learn mappings between function spaces rather than between individual solution instances. In contrast to classical surrogate models, which approximate a solution for a fixed discretization or geometry, neural operators aim to approximate the underlying solution operator itself. The theoretical foundations of this framework were established by Kovachki et al. (2021), who proved universal approximation results for neural operators in Banach spaces and unified several architecture families under a common kernel-integral perspective.

A particularly prominent representative is the Fourier Neural Operator (FNO) proposed by Z. Li et al. (2021). The FNO parameterizes integral kernel operators in the spectral domain by applying the Fast Fourier Transform, learning a truncated set of Fourier coefficients at each layer. This architecture offers two key advantages: first, it captures global, long-range dependencies in a single layer—a property that is naturally suited for problems governed by elliptic operators such as the Stokes equations; second, it achieves discretization invariance, meaning that a model trained on one grid resolution can, in principle, be evaluated on a different resolution without retraining. FNOs have demonstrated strong performance on benchmark PDE problems including Darcy flow, Burgers' equation, and two-dimensional Navier–Stokes turbulence (Z. Li et al. 2021).

An important architectural extension is the U-FNO proposed by Wen et al. (2022), which augments each Fourier layer with a U-Net-style branch to improve the representation of local, high-frequency features—a known limitation of the standard FNO arising from its truncation of high Fourier modes (Qin et al. 2024). Convolutional neural operators (Raonić et al. 2023) provide a further alternative, combining the discretization invariance of neural operators with spatially local convolution kernels. A comprehensive and fair comparison of FNO and DeepONet across multiple PDE benchmarks is provided by Lu et al. (2022), whose methodology also motivates the controlled comparison conducted in this thesis.

More broadly, convolution-based neural operators on structured grids have demonstrated the ability to generalize across variations in material parameters, forcing terms, and boundary conditions (Thuerey et al. 2020). These properties make neural operators appealing candidates for surrogate modeling in multiphase flow problems.

However, most existing studies focus on flows in relatively regular domains or on smoothly varying geometries. Applications to systems with many interacting rigid inclusions—such as sedimenting crystals with strong hydrodynamic coupling—remain scarce. In particular, the question of whether learned operators can generalize across changes in topological complexity, such as varying particle numbers $(1 \dots N)$, has not been systematically addressed. This gap is especially relevant for geophysical sedimentation problems,

where the number and spatial arrangement of crystals are inherently variable.

2.4 Machine Learning for Particle- and Crystal-Laden Flows

Fluid–particle interaction has long been an active area of research in classical computational fluid dynamics. Fully resolved numerical approaches, including lattice Boltzmann, immersed boundary, and fictitious-domain methods, have been used to study drafting–kissing–tumbling dynamics, clustering, and collective settling behavior in particulate flows (Ladd 1994; Leonardi et al. 2014). These methods provide detailed physical insight but are computationally expensive, particularly as the number of particles increases.

Machine-learning-based surrogates for particle- or crystal-laden flows are only beginning to emerge. Existing studies primarily focus on learning reduced quantities, such as drag corrections, closure terms, or accelerations within coupled CFD–DEM frameworks. Graph neural network (GNN) approaches have shown promising results for learning particle interaction dynamics from graph-structured representations of multi-particle systems (Sanchez-Gonzalez et al. 2020; Ma et al. 2022), and physics-informed deep learning has been applied to predict force distributions in dense suspensions of non-spherical particles (Ashwin et al. 2024). However, these methods typically target trajectory prediction or integrated force quantities rather than full spatial flow-field reconstruction. Surrogate models that directly map multi-particle geometric configurations to complete flow fields remain largely unexplored, especially in the low-Reynolds-number regime characteristic of magmatic systems. Numerical studies of crystal sedimentation in geophysical contexts further underscore the complexity of this regime and the need for efficient surrogate approaches (Verhoeven et al. 2009).

This absence of full-field surrogates is a notable limitation for applications where detailed spatial information is required, for example to analyze wake interactions, boundary-layer structures, or the emergence of collective sedimentation patterns.

2.5 Comparison of Network Paradigms for Flow-Field Surrogates

A wide range of machine-learning architectures has been proposed for approximating solutions of partial differential equations. For flow-field prediction problems, three paradigms are particularly relevant: coordinate-based implicit models, linear reduced-order models, and grid-based convolutional architectures.

2.5.1 Coordinate-Based Implicit Models

Coordinate-based neural networks represent physical fields as continuous functions of spatial coordinates. These models can achieve high accuracy and produce smooth reconstructions on fixed geometries. However, explicitly encoding complex and variable geometries within coordinate-based representations remains challenging. Extending such approaches to systems with multiple interacting inclusions typically requires substantial redesign of the input representation, limiting their scalability across varying particle numbers.

2.5.2 Linear Reduced-Order Models

Linear reduced-order models, such as Proper Orthogonal Decomposition or Principal Component Analysis, project high-dimensional flow fields onto a small number of dominant modes. While computationally efficient and interpretable, their linear nature fundamentally restricts their ability to capture nonlinear flow–geometry interactions. This limitation is particularly severe for multi-crystal Stokes flow, where wake interactions and boundary-layer effects play a central role.

2.5.3 Grid-Based Convolutional Surrogates

Grid-based convolutional neural networks operate directly on structured meshes and are therefore well suited for problems in which both inputs and outputs are defined on regular grids. U-Net architectures combine local feature extraction with global context through their encoder–decoder structure and skip connections (Ronneberger et al. 2015). A key advantage of this paradigm is its geometry-agnostic input representation: masks and distance fields can encode an arbitrary number of inclusions without requiring architectural modifications.

Previous studies demonstrate that CNN-based surrogates can generalize effectively across variable geometries and boundary conditions (Thuerey et al. 2020). These properties make grid-based convolutional models particularly attractive for multi-crystal sedimentation problems.

2.5.4 Spectral Neural Operators

Fourier Neural Operators (Z. Li et al. 2021) constitute an alternative grid-based paradigm that operates in the spectral rather than the spatial domain. Instead of learning localized convolutional kernels, FNOs learn global kernel functions parameterized by their Fourier coefficients. This spectral perspective provides inherent access to long-range correlations within a single network layer, which is advantageous for elliptic problems where local perturbations propagate throughout the entire domain.

A potential limitation of the standard FNO architecture is its reliance on low-frequency Fourier modes: the truncation of high-frequency coefficients may reduce accuracy near sharp interfaces such as crystal boundaries, where steep velocity gradients require high-frequency content (Qin et al. 2024). How this trade-off affects predictive performance in multi-crystal Stokes flow is one of the central questions investigated in this thesis.

2.5.5 Implications for This Work

The comparison of these paradigms reveals a trade-off between accuracy, geometric flexibility, and scalability. Coordinate-based implicit models excel on fixed geometries, and linear reduced-order models offer interpretability, but neither scales naturally to variable multi-inclusion configurations. Among grid-based approaches, U-Net and FNO architectures represent two complementary strategies: U-Nets capture multiscale spatial features through hierarchical encoder–decoder processing, while FNOs capture global structure through spectral convolutions (Lu et al. 2022). Directly comparing these two paradigms on the same multi-crystal Stokes flow problem is a core contribution of this thesis. Standardized PDE benchmark suites (Takamoto et al. 2022) provide evaluation protocols that inform the metric design adopted in this work.

2.6 Position of This Thesis in the Research Landscape

The reviewed literature reveals four key gaps:

1. The absence of surrogate models targeting multi-inclusion Stokes flow at the level of full flow fields.
2. A lack of systematic evaluation of surrogate-model generalization with respect to particle number (Morimoto et al. 2022).
3. Limited investigation of physically structured output representations—such as the stream function—as inductive biases in data-driven flow prediction.
4. No direct comparison of spatial (U-Net) and spectral (FNO) architectures for surrogate modeling of particle-laden Stokes flow with variable geometry (Lu et al. 2022).

This thesis addresses these gaps by constructing a dataset of multi-crystal Stokes simulations, predicting the stream function ψ as a physically structured learning target, and systematically comparing U-Net and FNO architectures with respect to generalization performance across varying crystal numbers and geometric configurations.

3 Theory and Physical Background

This chapter summarizes the physical, mathematical, and architectural foundations underlying the surrogate modeling approaches developed in this thesis. The first part covers low-Reynolds-number fluid mechanics and the stream-function formulation, which motivate the choice of learning target. The second part introduces the two neural network architectures compared in this work—the U-Net and the Fourier Neural Operator (FNO)—and discusses their structural properties in the context of flow-field prediction.

3.1 Incompressible Stokes Flow at Low Reynolds Number

Crystal settling in magmatic systems typically occurs at very low Reynolds numbers, where viscous forces dominate over inertial effects (Daniel Martin et al. 1988; Ladd 1994). In this creeping-flow regime, the governing equations reduce to the incompressible Stokes equations

$$\mathbf{0} = -\nabla p + \mu \nabla^2 \mathbf{u} + \rho \mathbf{g}, \qquad \nabla \cdot \mathbf{u} = 0, \tag{3.1}$$

where $\mathbf{u}$ denotes the velocity field, p the pressure, μ the dynamic viscosity, ρ the density, and $\mathbf{g}$ the gravitational acceleration.

A key property of Stokes flow is its linearity with respect to the velocity field for fixed material parameters and boundary conditions. This means that hydrodynamic interactions between rigid inclusions are long-ranged and additive: the disturbance induced by one particle influences the flow field throughout the domain. These long-range interactions form the physical basis for collective sedimentation phenomena, including wake interaction, drafting–kissing–tumbling dynamics, and clustering effects (Ladd 1994; Uhlmann et al. 2014).

In geophysical contexts, such interactions have been shown to strongly affect crystal settling rates and spatial distributions in magma chambers (Daniel Martin et al. 1988; Weinstein et al. 1988; D. Martin et al. 1989). Accurately capturing these effects therefore requires resolving the full flow field around multiple interacting crystals rather than relying on isolated particle approximations.

From a surrogate modeling perspective, the Stokes regime has two useful properties. First, the absence of inertia leads to smooth, steady-state flow fields for fixed crystal configurations. Second, incompressibility imposes a strong global constraint on admissible

velocity fields, which can be exploited through appropriate output representations.

3.2 Stream-Function Formulation in Two Dimensions

In two-dimensional incompressible flow, the divergence-free condition $\nabla \cdot \mathbf{u} = 0$ can be satisfied identically by introducing a scalar stream function $\psi(x, z)$ such that

$$u_x = \frac{\partial \psi}{\partial z}, \qquad u_z = -\frac{\partial \psi}{\partial x}. \tag{3.2}$$

This transformation reduces the vector-valued velocity field to a single scalar potential and guarantees incompressibility by construction.

The stream-function formulation has a long tradition in classical hydrodynamics, particularly for planar Stokes flow problems involving rigid inclusions and sedimentation (Ladd 1994; Uhlmann et al. 2014). Streamlines coincide with level sets of ψ, allowing qualitative flow features such as recirculation zones, wakes, and stagnation points to be identified directly.

From a numerical and learning perspective, the stream function offers several advantages. As a scalar field, ψ typically exhibits smoother spatial structure than individual velocity components, as it integrates information over spatial derivatives. Moreover, representing the solution in terms of ψ eliminates the need to explicitly enforce the incompressibility constraint during learning, in contrast to direct velocity-based approaches.

These properties motivate the use of the stream function as a learning target in this thesis, particularly in comparison to direct prediction of velocity components.

3.3 Vorticity and Poisson Reconstruction of the Stream Function

In two-dimensional incompressible flow, the scalar vorticity is defined as

$$\omega = \frac{\partial u_z}{\partial x} - \frac{\partial u_x}{\partial z}. \tag{3.3}$$

Taking the curl of the Stokes momentum equation eliminates the pressure term and yields a Poisson equation for the stream function,

$$\Delta \psi = -\omega. \tag{3.4}$$

This relationship provides a natural link between velocity-based numerical solutions and the stream-function representation. Given a divergence-free velocity field, the corresponding vorticity can be computed, and the stream function can be reconstructed by

solving a Poisson problem with appropriate boundary conditions. In the present work, this procedure is applied to high-fidelity LaMEM simulations to construct ψ as a learning target.

It is important to emphasize that this Poisson solve does not introduce new physics. Instead, it constitutes an auxiliary transformation that maps the numerical reference solution into a representation that is more amenable to learning. Similar transformations are commonly used in classical fluid mechanics to analyze Stokes flow around particles and inclusions (Ladd 1994).

3.4 Learning Targets as Physically Motivated Inductive Biases

From a machine-learning perspective, the choice of output representation introduces an implicit *inductive bias*, i.e. a structural constraint on the space of functions that the model can represent. In the context of surrogate modeling for incompressible flow, this choice strongly influences the degree to which physical structure must be learned from data rather than enforced analytically.

Direct velocity prediction treats the learning task as an unconstrained regression problem on a vector field. Incompressibility and momentum balance are not enforced explicitly but must be inferred from the training data. While this approach works well in many CNN-based flow surrogates (Ribeiro et al. 2021; Chen et al. 2019; Thuerey et al. 2020), it can lead to small but systematic violations of physical constraints.

Predicting the stream function ψ introduces a stronger inductive bias by restricting the hypothesis space to divergence-free velocity fields by construction. This approach is conceptually related to divergence-free neural representations proposed in the context of physics-informed and structure-preserving learning (Richter-Powell et al. n.d.), and is consistent with the neural operator framework (Kovachki et al. 2021) that views surrogate models as approximations of solution operators on function spaces. By reducing the effective degrees of freedom of the output, the learning problem is simplified and numerical conditioning can be improved.

Residual learning further modifies the inductive bias by decomposing the solution into a physically motivated baseline and a learned correction. If the baseline captures the dominant large-scale behavior of the flow, the residual field is typically smoother and of lower amplitude. Such strategies have been shown to improve learning efficiency and generalization in physics-based surrogate models (Sun et al. 2020).

3.5 U-Net Architecture

The U-Net (Ronneberger et al. 2015) is an encoder–decoder convolutional neural network originally developed for biomedical image segmentation. Its architecture consists of three principal components: a contracting encoder path, an expanding decoder path, and skip connections that bridge corresponding resolution levels.

3.5.1 Encoder Path

The encoder successively reduces the spatial resolution of the input through repeated application of convolutional layers followed by downsampling operations (typically max-pooling or strided convolutions). At each stage, the number of feature channels is increased, enabling the network to learn increasingly abstract and spatially coarse representations. For an input of spatial dimension $H \times W$, L encoding stages produce feature maps at resolutions $H/2^l \times W/2^l$ for $l = 1, \ldots, L$.

3.5.2 Decoder Path and Skip Connections

The decoder mirrors the encoder structure and progressively recovers spatial resolution through upsampling operations (transposed convolutions or interpolation) followed by convolutional layers. At each decoder stage, the upsampled feature map is concatenated with the corresponding encoder feature map via a skip connection. These skip connections are critical: they provide the decoder with high-resolution spatial information that would otherwise be lost during the encoding process, enabling the network to produce spatially precise outputs.

3.5.3 Multiscale Feature Extraction

The hierarchical structure of the U-Net endows it with an inherently multiscale receptive field. Shallow layers operate at fine spatial resolution and capture localized features such as boundary layers and sharp gradients near crystal surfaces. Deep layers operate at coarse resolution with large effective receptive fields and can represent domain-scale flow structures. The skip connections allow the final prediction to integrate information across all scales simultaneously.

For multi-crystal Stokes flow, this multiscale property is directly relevant: localized velocity gradients near crystal boundaries coexist with long-range hydrodynamic interactions that span the entire computational domain. The U-Net's ability to process both scales within a single forward pass makes it a natural candidate for this class of problems.

3.6 Fourier Neural Operator

The Fourier Neural Operator (FNO) (Z. Li et al. 2021) belongs to the family of neural operators that learn mappings between infinite-dimensional function spaces. Unlike conventional convolutional networks, which learn local spatial kernels, the FNO parameterizes integral kernel operators in the spectral domain.

3.6.1 Fourier Layer

The core building block of the FNO is the Fourier layer, which applies a linear transformation in frequency space. Given an input function $v(x) \in \mathbb{R}^{d_v}$ defined on a spatial domain, the Fourier layer computes

$$v_{l+1}(x) = \sigma\Big(W_l\, v_l(x) + \mathcal{F}^{-1}\big(R_l \cdot \mathcal{F}(v_l)\big)(x)\Big), \tag{3.5}$$

where $\mathcal{F}$ and $\mathcal{F}^{-1}$ denote the Fast Fourier Transform and its inverse, $R_l \in \mathbb{C}^{k_{\max} \times d_v \times d_v}$ is a learnable weight tensor applied to the lowest $k_{\max}$ Fourier modes, W_l is a pointwise linear transformation (bias path), and σ is a nonlinear activation function.

The key operation is the multiplication $R_l \cdot \mathcal{F}(v_l)$ in Fourier space, which is equivalent to a global convolution in physical space. By retaining only $k_{\max}$ modes, the FNO implicitly applies a low-pass filter while keeping the number of learnable parameters independent of the spatial discretization.

3.6.2 Architecture

A complete FNO model consists of three stages:

1. A pointwise lifting layer $P : \mathbb{R}^{d_a} \to \mathbb{R}^{d_v}$ that projects the input channels into a higher-dimensional feature space.

2. A sequence of L Fourier layers as defined in Equation (3.5), each combining spectral and local processing.

3. A pointwise projection layer $Q : \mathbb{R}^{d_v} \to \mathbb{R}^{d_o}$ that maps the learned features to the desired output dimension.

3.6.3 Properties Relevant to Stokes Flow

The FNO architecture has several properties that are relevant for the surrogate modeling task considered in this work:

- **Global receptive field.** Each Fourier layer captures dependencies across the entire spatial domain in a single operation. For Stokes flow, where the elliptic character

of the governing equations implies that local perturbations propagate globally, this is a structurally natural property.

- **Spectral bias.** The truncation to $k_{\max}$ Fourier modes introduces an implicit smoothness prior. While this favors the representation of large-scale flow structures, it may limit accuracy near sharp interfaces such as crystal boundaries, where high-frequency content is physically significant (Qin et al. 2024).

- **Discretization invariance.** Because the FNO operates on continuous function representations via their Fourier coefficients, a model trained on one grid resolution can, in principle, be evaluated on a different resolution without retraining.

3.7 Architectural Comparison: U-Net vs. FNO

The U-Net and FNO embody fundamentally different inductive biases for structured prediction tasks. Table 3.1 summarizes the key distinctions.

Table 3.1: Structural comparison of U-Net and FNO architectures.

Property	**U-Net**	**FNO**
Domain of operation	Spatial	Spectral (Fourier)
Receptive field	Hierarchical, local-to-global	Global per layer
Multiscale mechanism	Encoder–decoder + skip connections	Mode truncation
High-frequency content	Preserved via skip connections	Limited by $k_{\max}$
Parameter scaling	Depends on depth and channels	Independent of resolution

For multi-crystal Stokes flow, these differences translate into complementary strengths: the U-Net is expected to excel at resolving sharp boundary layers and localized wake structures due to its hierarchical spatial processing, while the FNO may offer advantages in capturing the global, elliptic character of Stokes interactions through its spectral representation. The empirical comparison of these architectures on identical training data and evaluation metrics is a central contribution of this thesis.

3.8 Implications for Multi-Crystal Sedimentation Modeling

The theoretical considerations discussed above have direct implications for surrogate modeling of multi-crystal sedimentation. Long-range hydrodynamic interactions and incompressibility impose global structure on the flow field, which is difficult to capture through purely local regression of velocity components. Predicting the stream function ψ provides

a principled way to reduce the complexity of the learning task by enforcing incompressibility analytically.

The choice of network architecture interacts with this physical structure in complementary ways. The U-Net's hierarchical encoder–decoder processing naturally mirrors the multiscale character of the flow: sharp boundary layers near crystal surfaces are captured at fine resolution levels, while domain-scale circulation patterns are represented in the deep, coarse layers. The FNO's spectral approach, on the other hand, directly reflects the elliptic nature of the Stokes equations, where local perturbations propagate globally—a property that is inherently encoded in the Fourier representation.

These complementary architectural biases are what motivates the comparison pursued in this thesis: by training both architectures on the same data and evaluating them under identical conditions, we can disentangle the effects of physical output representation (stream function) from those of architectural design (spatial vs. spectral processing).

The next chapter builds on this theoretical foundation by translating these considerations into a concrete computational methodology for data generation, network design, training, and evaluation.

4 Methodology

This chapter describes the computational workflow for generating training data, constructing machine-learning inputs, training surrogate models, and evaluating their generalization performance. The methodology builds on the literature reviewed in Chapter 2 and provides a reproducible end-to-end pipeline for structured-grid flow-field prediction.

Both surrogate architectures considered in this thesis—U-Net and Fourier Neural Operator (FNO)—predict the stream function ψ on a fixed 256×256 grid. The velocity field is then recovered analytically from ψ via its spatial derivatives, guaranteeing incompressibility by construction. The simulation setup, input encoding, optimization procedure, and evaluation protocol are kept identical for both architectures, so that performance differences can be attributed to the architectural design—spatial hierarchical processing (U-Net) versus spectral global processing (FNO)—rather than to confounding factors in data or training.

4.1 Learning Target: Stream-Function Prediction

Both surrogate architectures predict the scalar stream function $\psi(x, z)$ on the computational grid. The velocity field is recovered analytically via

$$\mathbf{u} = \left(\frac{\partial \psi}{\partial z}, \; -\frac{\partial \psi}{\partial x} \right),$$

which enforces incompressibility by construction. The theoretical motivation for this formulation and its advantages over direct velocity prediction were discussed in Chapter 3.

From the perspective of surrogate modeling, predicting ψ instead of $\mathbf{u}$ directly offers two key benefits. First, it reduces the learning problem from a vector-valued to a scalar regression task, lowering the effective degrees of freedom. Second, incompressibility is guaranteed analytically rather than having to be learned from data, which eliminates a common source of systematic error in purely data-driven velocity surrogates (Ribeiro et al. 2021; Thuerey et al. 2020). This choice injects physical structure into the learning problem through the output representation rather than through penalty terms, consistent with divergence-free parameterizations advocated in related work (Richter-Powell et al. n.d.).

The central experimental factor in this thesis is therefore not the choice of learning

target—which is fixed to ψ for all experiments—but the choice of network architecture. By training both U-Net and FNO on identical data with the same learning target, performance differences can be attributed to the architectural inductive bias: spatial hierarchical processing versus spectral global processing.

4.2 LaMEM Simulations for Data Generation

Training data are generated using the geodynamic finite-element code LaMEM, which solves the incompressible Stokes equations on a regular Cartesian grid. The simulation setup follows established geodynamic Stokes discretization practices, where robustness under viscosity and density contrasts is essential (Thieulot et al. 2022).

Each simulation represents a steady (quasi-static) configuration of $1 \dots N_{\max}$ rigid crystals settling in a viscous matrix. Simulations are advanced until a quasi-steady velocity field is obtained; transient effects are not considered further, as the surrogate models are trained to predict steady-state fields.

4.2.1 Domain and Discretization

The computational domain is a rectangular box discretized on a fixed 256×256 grid with uniform spacing $\Delta x = \Delta z$. The surrogate models operate on fields represented on this grid. While physical units are retained internally during simulation, learning is performed on consistently scaled grid-based fields, and the fixed discretization provides a controlled benchmark for comparing output representations.

Crystals are modeled as rigid circular inclusions with prescribed radii and randomized positions. The number of crystals N is drawn uniformly from $\{1, \dots, N_{\max}\}$ to expose the surrogate to a broad range of interaction regimes, from isolated single-crystal wakes to strongly interacting clusters. Such multi-particle interaction effects are well documented in classical sedimentation studies (Ladd 1994) and motivate an explicit evaluation of generalization across particle number. Randomization is performed independently for each simulation run, such that the training distribution samples a diverse subset of the configuration space.

4.2.2 Material Properties and Governing Equations

The matrix phase is assigned viscosity μ and density ρ_f. Crystals are approximated as effectively rigid particles by assigning a viscosity several orders of magnitude larger than μ and a density $\rho_p > \rho_f$ to induce gravitational settling. Within LaMEM, this results in a two-phase Stokes problem with sharp viscosity and density contrasts.

The governing equations are the incompressible Stokes equations

$$\mathbf{0} = -\nabla p + \mu \nabla^2 \mathbf{u} + \rho \mathbf{g}, \qquad \nabla \cdot \mathbf{u} = 0,$$

solved with free-slip boundary conditions on all walls. Free-slip is enforced by vanishing normal velocity and vanishing tangential shear stress at the boundaries, reducing artificial boundary-layer effects and providing a simple, well-posed benchmark setting.

4.2.3 Extraction of Flow Quantities

For each simulation, LaMEM outputs grid-based fields including the phase indicator and the velocity components as well as velocity gradients:

$$\{\text{phase field}, u_x, u_z, \nabla u_x, \nabla u_z\}.$$

The phase field indicates whether a cell belongs to the crystal or matrix.

From the velocity gradients, the scalar vorticity is computed as

$$\omega = \frac{\partial u_z}{\partial x} - \frac{\partial u_x}{\partial z},$$

consistent with the two-dimensional convention used in Chapter 3. The vorticity field serves as the right-hand side for reconstructing the stream function ψ via a Poisson solve, as described in Section 4.3.

4.3 Computing and Normalizing the Stream Function

The stream function is reconstructed from vorticity by solving the Poisson equation

$$\Delta \psi = -\omega$$

on the same 256×256 grid. Reconstructing ψ via a Poisson problem is standard in two-dimensional incompressible low-Reynolds-number flow theory. In the present workflow, this Poisson solve is used only to define a physically consistent scalar learning target; the underlying reference solution remains the LaMEM Stokes solution.

4.3.1 Discretization and Boundary Conditions

The Laplace operator is discretized with a second-order five-point stencil on interior grid points. Homogeneous Dirichlet conditions $\psi|_{\partial\Omega} = 0$ are imposed on the domain boundary to fix the stream-function gauge and yield a unique discrete solution. Since all evaluation comparisons are ultimately performed against LaMEM-derived quantities on the

same grid, the Poisson solve serves as an auxiliary reconstruction step rather than an independent physical model.

The resulting sparse linear system is solved iteratively. The specific solver is not critical for the learning setup, provided the reconstruction error is small compared to the surrogate prediction error.

4.3.2 Motivation and Procedure for Normalization

Raw values of ψ can span several orders of magnitude (typically 10^{-16} to 10^{-10}), depending on velocity magnitude, viscosity, and grid spacing. While such values are well resolved in the numerical solver, they are unfavorable for gradient-based learning because they lead to ill-conditioned losses and large sample-to-sample dynamic range. To improve numerical conditioning, each sample is normalized individually. We compute a characteristic order of magnitude

$$p_{\text{mean}} = \text{round}(\text{mean}\left(\log_{10}(|\psi| + \epsilon)\right)), \qquad \epsilon = 10^{-30},$$

and scale the field as

$$\psi_{\text{norm}} = \psi \cdot 10^{-p_{\text{mean}}}.$$

Training is performed on ψ_{norm}, and predictions are rescaled after inference by multiplying with $10^{p_{\text{mean}}}$. This preserves the spatial structure and sign of ψ while shifting values into an $\mathcal{O}(1)$ range that yields stable optimization.

4.4 Input Representation: Mask, Distance Field, and Coordinates

Both architectures receive the same five-channel input, generated by a shared data pipeline. Training data are generated using the FNO data generation code and stored as JLD2 files; these files are then loaded identically for both U-Net and FNO training. Encoding geometry through masks, distance fields, and coordinate channels is common in structured-grid PDE surrogates and geometry-aware learning frameworks (Thuerey et al. 2020; Oldenburg et al. 2022).

The five input channels are:

1. **Crystal mask.** A binary indicator field $M(x, z) \in \{0, 1\}$ specifying crystal locations:

$$M(x, z) = \begin{cases} 1, & \text{inside any crystal,} \\ 0, & \text{in the matrix.} \end{cases}$$

   This channel encodes the location of rigid inclusions and material contrasts.

2. **Signed distance field (SDF).** For each grid point, the signed distance to the nearest crystal boundary is computed: negative inside crystals, positive outside. This provides a smooth measure of boundary proximity that complements the sharp binary mask and helps the network resolve steep velocity gradients near solid–fluid interfaces.

3. **Radius of nearest crystal.** For each grid point, the radius of the geometrically closest crystal is stored as an explicit channel. In Experiments 1–3, all crystals share the same fixed radius, so this channel carries no configurational information and has little effect on model behavior. It is included to prepare the input representation for Experiment 4, in which crystal radii vary and this channel becomes a meaningful geometric feature.

4. **Normalized x-coordinate.** The horizontal coordinate is linearly mapped to $x_{\text{norm}} \in [-1, 1]$, providing the network with absolute positional information.

5. **Normalized z-coordinate.** Analogously, the vertical coordinate is mapped to $z_{\text{norm}} \in [-1, 1]$. Together, the coordinate channels reduce symmetry-induced and position-dependent artifacts.

Each sample is represented by an input tensor of shape $(256, 256, 5)$. The input encodes inclusion geometry, proximity to crystal boundaries, and domain position, and is identical for both architectures.

4.5 Surrogate Architectures

Two architectures are compared in this thesis. Both receive the same five-channel input of shape $(256, 256, 5)$ and produce a single-channel output ψ_{norm} of shape $(256, 256, 1)$. The theoretical foundations of both architectures were introduced in Chapter 3; this section describes the specific configurations used in the experiments.

4.5.1 U-Net

The U-Net follows the encoder–decoder principle of the original architecture (Ronneberger et al. 2015) and common adaptations for laminar flow-field prediction on Cartesian grids (Thuerey et al. 2020; Chen et al. 2019).

Encoder. The encoder consists of four resolution levels. Spatial resolution is reduced by a factor of two at each level, while feature widths increase as $32 \to 64 \to 128 \to 256$. Each level contains two 3×3 convolutions with batch normalization and ReLU activations, followed by downsampling via a strided convolution (stride 2). Strided downsampling is

used instead of max pooling to keep the operation learnable and to reduce grid-aligned artifacts in smooth Stokes-flow targets.

Bottleneck. At the coarsest resolution, a bottleneck block with two 3×3 convolutions encodes global flow structure conditioned on the entire crystal configuration. This compressed representation provides the long-range context required to model hydrodynamic interactions between distant inclusions.

Decoder. The decoder mirrors the encoder: transposed convolutions upsample the feature maps, which are then concatenated with encoder activations through skip connections. Each decoding stage applies a two-layer 3×3 convolutional block with batch normalization and ReLU activation. Skip connections preserve high-resolution geometric detail, which is essential to reconstruct sharp structures near crystal boundaries while maintaining global consistency.

Output layer. A final 1×1 convolution projects the decoder features onto a single output channel (ψ_{norm}). No activation is applied in the final layer, as the task is a continuous regression problem.

4.5.2 Fourier Neural Operator

The FNO follows the architecture proposed by Z. Li et al. (2021), consisting of a pointwise lifting layer, a sequence of Fourier layers, and a pointwise projection layer.

Lifting. A pointwise linear layer maps the five input channels into a higher-dimensional feature space of width d_v.

Fourier layers. Each Fourier layer applies the operation defined in Equation (3.5): the input is transformed into Fourier space via a two-dimensional FFT, multiplied with a learnable complex weight tensor R_l for the lowest $k_{\max}$ modes, and transformed back. This spectral path is combined with a pointwise linear transformation (bias path) and a nonlinear activation. A stack of L such layers forms the core of the model.

Projection. A pointwise linear layer maps the learned feature representation to the single output channel ψ_{norm}.

Hyperparameters. The specific values of d_v, $k_{\max}$, and L are reported together with the experimental results in Chapter 7, where they are chosen to yield a model with comparable parameter count to the U-Net to ensure a fair comparison.

4.5.3 Regularization and Practical Considerations

Both architectures are deliberately kept lightweight to enable repeated training runs on standard GPU hardware. For the U-Net, batch normalization stabilizes optimization. For the FNO, the implicit spectral smoothness prior imposed by mode truncation serves as an architectural regularizer. Generalization is primarily promoted through geometric diversity in the pre-generated training data and the physically structured output representation. Dropout is not used, as the large variety of randomized crystal configurations in the training set provides sufficient regularization.

4.6 Training Procedure

4.6.1 Data Splits and Benchmark Design

The dataset is divided into three fixed splits—training, validation, and test—each generated once from randomized crystal configurations and stored on disk prior to any model training. All three splits are identical for both the U-Net and FNO pipelines, ensuring that performance differences between architectures are not confounded by differences in training or evaluation data. The validation split is used for monitoring during training, while the test split provides a reproducible benchmark for the final evaluation.

4.6.2 Batching and Device Handling

Samples are stored in `.jld2` format to enable efficient loading of multi-channel arrays in Julia. Mini-batches are assembled on the fly, with batch size chosen as a compromise between gradient stability and GPU memory constraints. Data tensors and model parameters are explicitly moved to CPU or GPU to ensure consistent device placement and portable execution.

4.6.3 Loss Function

The primary loss is the mean-squared error

$$\mathcal{L}_{\mathrm{MSE}} = \frac{1}{N_{\mathrm{pix}}} \sum_i \left(y_i^{\mathrm{pred}} - y_i^{\mathrm{true}} \right)^2 .$$

In addition, the implementation supports Huber loss and weighted MSE to increase robustness in regions with sharp gradients or to emphasize accuracy near inclusion boundaries. The present study uses a consistent baseline loss for both architectures to ensure a controlled comparison.

4.6.4 Optimization and Checkpointing

Model parameters are optimized using the Adam optimizer with adaptive learning rates. A fixed learning rate is used within each run, without learning rate scheduling, to keep experiments comparable. After each epoch, model checkpoints are stored in a CPU-compatible format to enable reproducible evaluation on systems without GPU support. For each run, training configuration metadata (hyperparameters, data paths, and random seeds) are archived alongside the checkpoint.

4.7 Evaluation Pipeline

Model predictions are evaluated on the fixed test dataset containing crystal geometries not used during training. This separation ensures that evaluation captures generalization to configurations not seen during training.

Primary evaluation metrics include:

- mean-squared error (MSE) of the predicted target field,
- relative L_2 errors for ψ, $\partial_x \psi$, and $\partial_z \psi$, which quantify errors in both amplitude and derived velocity gradients,
- pixelwise error thresholds $\epsilon_{0.01}$, $\epsilon_{0.05}$, and $\epsilon_{0.10}$, defined as the fraction of grid points exceeding relative error levels of 1%, 5%, and 10%.

Errors are grouped by the number of crystals in the test sample ($1 \ldots N_{\max}$), enabling direct assessment of how prediction quality changes as hydrodynamic interactions become more complex—an approach motivated by generalization studies that stratify performance across geometric complexity (Morimoto et al. 2022). The use of MSE and relative norms follows common practice in benchmarking data-driven CFD surrogates (Takamoto et al. 2022; Lu et al. 2022). The same evaluation framework is applied identically to both U-Net and FNO predictions, ensuring that all reported differences reflect architectural properties rather than evaluation artifacts.

4.8 Summary

This chapter described the full surrogate modeling workflow: LaMEM-based data generation, reconstruction and normalization of the stream function, input encoding, U-Net and FNO architecture configurations, training procedure, and evaluation protocol. The modular design ensures that all pipeline components except the network architecture are kept identical for both models, so that performance differences can be attributed to the architectural inductive bias rather than to differences in data or training. The next chapter describes the technical implementation of this pipeline.

5 Implementation

This chapter describes the technical realization of the surrogate modeling framework introduced in Chapter 4. While the previous chapter motivated the modeling choices—learning targets, input encoding, and architecture—the focus here is on the concrete implementation of the full workflow in a reproducible and modular software stack.

All components are implemented in Julia and designed to be hardware-agnostic: the same code paths run on CPU and GPU without modification. Wherever possible, explicit implementations are used instead of high-level abstractions to retain full control over numerical behavior, memory usage, and device placement.

5.1 Software Environment

All experiments are implemented in **Julia 1.10.4**. The core dependencies are:

- **Flux.jl** (v0.16.5) for neural network layers, automatic differentiation, and parameter handling.
- **CUDA.jl** (v5.9.3) for GPU acceleration on CUDA-capable hardware.
- **NNlib.jl** for low-level neural network operations, in particular batched matrix multiplication used in the FNO spectral convolution layers.
- **FFTW.jl** for the Fast Fourier Transform operations required by the Fourier Neural Operator.
- **Optimisers.jl** for optimizer definitions, in particular Adam (U-Net) and Adam with gradient clipping (FNO).
- **JLD2.jl** for efficient storage and loading of multi-channel training samples.
- **BSON.jl** for serializing and restoring model checkpoints.
- **PyPlot.jl / CairoMakie.jl** for visualization of predicted and reference fields during evaluation.

At runtime, CUDA availability is detected automatically. If a compatible GPU is present, arrays and model parameters are moved to the GPU; otherwise, the implementation falls back transparently to CPU execution.

5.2 Code Organization and Execution Flow

The code base is organized into two parallel module trees that share common infrastructure but implement architecture-specific components independently:

- `UNET_Ansatz/` contains the U-Net model, training pipeline, and evaluation routines.
- `FNO_Ansatz/` contains the FNO model, spectral convolution layers, composite loss functions, and its own training and evaluation pipelines.

Both trees share the same LaMEM interface module for data generation, the same Poisson solver for stream-function reconstruction, and the same dataset interface (`PsiDataset`, `batch_iterator`), ensuring that U-Net and FNO train and are evaluated on identical data. Within each tree, a main execution script dispatches to dedicated modules depending on the selected workflow stage: data generation, preprocessing, training, or evaluation. Each stage can be executed independently, enabling flexible experimentation and reducing coupling between components.

5.3 Model Implementations

Both architectures are implemented explicitly using Flux primitives. They share the same input and output interface: a five-channel input tensor of shape $(256, 256, 5)$ and a single-channel output ψ_{norm} of shape $(256, 256, 1)$. Both models load and process the same pre-generated JLD2 datasets; the number of input channels is determined dynamically at model construction time from the first loaded sample. This makes the representation geometry-agnostic: configurations with varying numbers of crystals are encoded on the same fixed grid without architectural changes.

U-Net

The encoder–decoder structure follows a classical U-Net design with skip connections. Downsampling uses strided convolutions instead of max pooling to keep the operation learnable and reduce grid-alignment artifacts in smooth flow fields. Each resolution level applies two successive 3×3 convolutions, followed by batch normalization and ReLU activation.

Upsampling is implemented via bilinear interpolation followed by two successive 3×3 convolutions, avoiding the checkerboard artifacts that can arise from transposed convolutions. Skip connections concatenate encoder feature maps with decoder feature maps at matching resolutions. A final 1×1 convolution maps the decoder output to the single output channel with no activation, as the task is pure regression. The model is constructed via a `build_unet()` factory function with configurable base channel width; the concrete value is reported in Chapter 6.

Fourier Neural Operator

The FNO implementation consists of two custom layer types and a model constructor:

SpectralConv2D. The spectral convolution layer implements the frequency-space multiplication described in Chapter 3. It uses a real-valued FFT (`rfft`) instead of a full complex FFT, reducing memory consumption by approximately 50%. Two sets of complex-valued weight matrices $W_1, W_2 \in \mathbb{C}^{k_{\max} \times d_v \times d_v}$ are learned for positive and negative frequency components along the vertical axis. The forward pass applies a batched matrix multiplication in Fourier space via `NNlib.batched_mul`, which is compatible with Zygote's automatic differentiation (no in-place array mutations). Weights are initialized with small magnitude ($\sigma = 0.02$) to provide implicit warmup during early training.

FNOBlock. Each FNO block combines the spectral convolution path with a pointwise 1×1 convolution (local bias path). Both outputs are added element-wise and passed through a GELU activation function. This parallel-path design allows each block to simultaneously capture global spectral structure and local spatial features.

Model assembly. The complete model chains a pointwise lifting convolution ($4 \to d_v$ channels), a stack of L FNO blocks, and a two-layer pointwise projection ($d_v \to d_v$, GELU, $d_v \to 1$). The architecture is parameterized by the channel width d_v, the number of retained Fourier modes $k_{\max}$, and the number of Fourier layers L; concrete values are reported in Chapter 6.

5.4 Data Pipeline and Dataset Interfaces

Training samples are stored as individual `.jld2` files. Each file contains:

- the input tensor,
- the normalized target field,
- metadata describing crystal configuration and grid parameters.

All arrays are stored and processed in single precision (`Float32`) to reduce memory usage and improve GPU throughput, without measurable loss of accuracy for the present regression task.

Dataset abstraction and batching

Custom dataset types provide fine-grained control over data loading and batch assembly. Rather than relying on high-level data loaders, batches are constructed explicitly to min-

imize memory overhead and to enable dynamic adjustment of batch size. The dataset interface supports:

- random shuffling of samples,
- variable batch sizes (typically between 2 and 16),
- transparent transfer of batches to CPU or GPU.

LaMEM-based data generation is executed serially. This avoids race conditions and ensures deterministic behavior in the presence of a solver that is not thread-safe.

5.5 Stream-Function Reconstruction and Normalization

For stream-function learning, the vorticity field produced by LaMEM is converted into a stream function by solving the Poisson equation

$$\Delta\psi = -\omega$$

on the same Cartesian grid using a finite-difference discretization with homogeneous Dirichlet boundary conditions. This reconstruction step is used exclusively to define the learning target and is not part of the inference pipeline.

The reconstructed stream function is normalized on a per-sample basis to shift values into an $\mathcal{O}(1)$ range. The network is trained only on the normalized field ψ_{norm}, while rescaling to physical units is applied during post-processing and evaluation. This prevents extremely small target magnitudes from approaching machine precision and improves numerical conditioning of the regression loss.

5.6 Training Loop and Optimization

Training is implemented using an explicit loop to retain full control over gradient computation, parameter updates, checkpointing, and device placement. Gradients are computed via Flux's automatic differentiation.

Optimizer configuration

Both models use the Adam optimizer from `Optimisers.jl`. The FNO additionally applies gradient norm clipping (`ClipNorm`) to stabilize training, as spectral operations can produce large gradient magnitudes. Concrete hyperparameter choices—learning rates, gradient clipping thresholds, batch sizes, and number of training epochs—are architecture-specific and reported in Chapter 6.

Loss functions

Both U-Net and FNO are trained with the same composite loss

$$\mathcal{L} = \mathcal{L}_{\text{MSE}} + \alpha_{\text{grad}} \cdot \mathcal{L}_{\text{grad}} + \alpha_{\text{bnd}} \cdot \mathcal{L}_{\text{bnd}},$$

where $\mathcal{L}_{\text{grad}}$ penalizes errors in the spatial derivatives $\partial\psi/\partial x$ and $\partial\psi/\partial z$ (computed via finite differences), and $\mathcal{L}_{\text{bnd}}$ enforces the homogeneous Dirichlet boundary condition $\psi|_{\partial\Omega} = 0$. The gradient loss coefficient α_{grad} is ramped from zero over a configurable number of warmup epochs to avoid destabilizing early optimization. This composite loss encourages both models to produce predictions that are not only accurate in ψ itself but also yield physically consistent velocity fields and respect the boundary conditions.

Checkpointing

U-Net checkpoints are written at the end of each epoch in BSON format, which ensures CPU-compatible portability. FNO checkpoints use JLD2 and additionally store the optimizer state and epoch index, enabling training to be resumed from any saved checkpoint. In both cases, model parameters are explicitly transferred to CPU before saving.

5.7 Evaluation Utilities

Both implementations include dedicated evaluation toolboxes that support quantitative benchmarking and qualitative diagnostics. Quantitative metrics include mean-squared error, mean absolute error, relative L_2 norms, and maximum absolute error, computed on the stream function ψ as well as on derived velocity fields obtained via $\mathbf{u} = (\partial\psi/\partial z,\ -\partial\psi/\partial x)$. The FNO evaluation additionally reports gradient MSE and boundary MSE to assess the quality of the composite loss components. Errors are aggregated by crystal count to assess generalization as a function of geometric complexity.

Qualitative diagnostics include side-by-side visualizations of predicted and reference fields, difference plots, and derived velocity representations such as quiver and streamline plots. All evaluation routines operate independently of the training code and can be applied to any stored checkpoint, enabling systematic comparison between U-Net and FNO across training runs.

5.8 Summary

This chapter described the concrete implementation of the surrogate modeling framework, including the software environment, code organization, U-Net and FNO realizations, dataset interfaces, training loop, and evaluation utilities. Key implementation

decisions—such as the use of `rfft` and Zygote-compatible operations in the FNO, and the composite loss with gradient and boundary terms—are motivated by the specific requirements of stream-function prediction for multi-crystal Stokes flow. Together with the methodology in Chapter 4, this implementation provides a reproducible basis for the numerical experiments presented in the following chapters.

6 Experiments

This chapter describes the experimental setup used to evaluate and compare the U-Net and FNO surrogate models. It covers the dataset and data split, the learning task and input encoding, the evaluation metrics, and the specific experimental configurations designed to assess model accuracy and out-of-distribution generalization.

6.1 Dataset and Data Split

All training and evaluation data are generated using LaMEM, a parallel finite-difference Stokes solver. Each simulation produces a steady-state Stokes flow in a $[-1, 1]^2$ domain containing one or more circular crystals with fixed viscosity contrast. The stream function ψ is reconstructed from the vorticity output via the Poisson solver described in Chapter 5, and normalized per sample before storage.

Samples are stored as individual `.jld2` files, each containing the input tensor, the normalized target field ψ_{norm}, the normalization scale, and grid metadata. Data generation is executed serially to guarantee deterministic behavior.

The dataset is stratified by crystal count. For each stratum, 1 000 samples are used for training, 100 for validation, and 10 for final evaluation. Samples with $n = 1, 2, 3$ crystals are used for training and validation; generalization to higher crystal counts is assessed on held-out test samples. The 1 000 / 100 training-to-validation ratio is maintained within each stratum, ensuring balanced representation across all geometric complexity levels.

6.2 Learning Task and Input Encoding

Both models learn the mapping from a geometric encoding of the crystal configuration to the normalized stream function ψ_{norm}. The stream function is chosen as the learning target because it implicitly encodes the incompressibility constraint: the velocity field $\mathbf{u} = (\partial\psi/\partial z,\ -\partial\psi/\partial x)$ is divergence-free by construction.

Each input sample consists of four channels on a 256×256 Cartesian grid:

1. **Crystal mask** — binary indicator of solid crystal regions (1 inside, 0 outside).

2. **Signed distance field (SDF)** — signed distance to the nearest crystal boundary, providing smooth geometric information beyond the binary mask.

3. **Normalized x-coordinate** — grid position along the horizontal axis, scaled to $[-1, 1]$.

4. **Normalized z-coordinate** — grid position along the vertical axis, scaled to $[-1, 1]$.

The coordinate channels allow the network to resolve position-dependent flow patterns without relying on positional embeddings. The single output channel is $\psi_{\text{norm}} \in \mathbb{R}^{256\times256\times1}$; rescaling to physical units is applied in post-processing using the stored per-sample scale factor.

6.3 Evaluation Metrics

Predictions are evaluated on the normalized stream function ψ_{norm} as well as on derived velocity fields $\mathbf{u} = (\partial\psi/\partial z, -\partial\psi/\partial x)$, computed via finite differences on the 256×256 grid. The following metrics are reported:

MSE Mean squared error on ψ_{norm}, measuring overall pointwise accuracy.

Relative L_2 error $\|\hat{\psi} - \psi\|_2/\|\psi\|_2$, providing a scale-invariant measure of generalization quality.

Maximum absolute error $\|\hat{\psi} - \psi\|_\infty$, quantifying worst-case local deviations.

Velocity MSE MSE on the derived velocity components u_x and u_z, assessing the physical consistency of the predicted stream function beyond pointwise ψ accuracy.

Gradient MSE MSE on the spatial derivatives $\partial\psi/\partial x$ and $\partial\psi/\partial z$, directly reflecting the gradient loss term used during training.

Boundary MSE MSE evaluated exclusively on the domain boundary $\partial\Omega$, measuring compliance with the homogeneous Dirichlet condition $\psi|_{\partial\Omega} = 0$.

All metrics are aggregated by crystal count to assess how performance scales with geometric complexity. This stratification is essential for distinguishing in-distribution accuracy from out-of-distribution generalization.

6.4 Experimental Configurations

Four experiments are conducted to systematically evaluate and compare both models. The same standard architecture configurations are used across all experiments; key hyperparameters are summarized in Table 6.1 in Section 6.5.

Experiment 1 – Single-crystal baseline. Both models are trained and evaluated on configurations with exactly one crystal ($n = 1$). This experiment establishes a baseline for each architecture under the simplest geometric setting and provides a controlled comparison of their accuracy and training behavior. Four learning rates are evaluated (1×10^{-3}, 5×10^{-3}, 1×10^{-4}, 5×10^{-4}), each combined with batch sizes of 8 and 16, and results are compared systematically.

Experiment 2 – Multi-crystal generalization. Models are trained on samples with $n \in \{1, \ldots, 10\}$ crystals and evaluated separately for each crystal count, including unseen higher counts ($n = 11, \ldots, 25$). This experiment assesses the ability of each architecture to generalize to geometric configurations more complex than those encountered during training—a capability that is critical for practical applications in geodynamics, where the number of crystals can vary widely.

Experiment 3 – Stress-testing generalization limits. Models are trained on configurations with up to $n = 25$ crystals and subsequently evaluated on substantially more complex scenes with up to $n = 100$ crystals. This experiment probes whether the models can capture the interactions arising from a large number of obstacles or whether their generalization capability breaks down beyond a critical level of geometric complexity.

Experiment 4 – Crystal size generalization. Models are trained as in Experiment 2, but the crystals in the evaluation samples are additionally scaled by up to a factor of ten in both directions relative to the training distribution. This experiment tests whether the models can generalize not only to larger numbers of crystals, but also to unseen obstacle sizes, introducing a further axis of out-of-distribution generalization.

6.5 Training Setup and Hyperparameters

Both models are trained on the same datasets and evaluated with the same metrics. Table 6.1 summarizes the key hyperparameters.

Loss functions. Both architectures are trained with the same composite loss that combines a pointwise MSE term with a gradient-matching term and a boundary term:

$$\mathcal{L} = \mathrm{MSE}(\hat{\psi}, \psi) + \alpha_{\mathrm{grad}}\, \mathcal{L}_{\mathrm{grad}} + \alpha_{\mathrm{bnd}}\, \mathcal{L}_{\mathrm{bnd}}, \tag{6.1}$$

where $\mathcal{L}_{\mathrm{grad}} = \mathrm{MSE}(\partial_x \hat{\psi}, \partial_x \psi) + \mathrm{MSE}(\partial_z \hat{\psi}, \partial_z \psi)$ is computed via central differences on interior points, and $\mathcal{L}_{\mathrm{bnd}} = \mathrm{MSE}(\hat{\psi}|_{\partial\Omega}, 0)$ enforces the homogeneous Dirichlet condition. The coefficient α_{grad} is ramped linearly from 0 to 0.1 over the first five epochs to stabilize

early training; $\alpha_{\text{bnd}} = 0.01$ is kept fixed. The MSE term is evaluated on interior grid points only (a two-pixel boundary strip is excluded via a mask).

The gradient term serves two purposes: it directly penalizes errors in the spatial derivatives of ψ, which propagate into the derived velocity field, and it acts as an implicit regularizer that suppresses high-frequency residuals. Preliminary experiments with a plain MSE loss for the U-Net showed substantially elevated velocity errors relative to the ψ error (a factor of roughly 3×); adopting the composite loss reduced velocity errors significantly while leaving ψ accuracy largely unchanged for batch size 8 (see Section 7.1.5).

Model selection. Both models are checkpointed after every epoch; the checkpoint with the lowest validation MSE is selected as the final model.

Hardware. All experiments are run on a single NVIDIA A100 GPU. Models are saved in CPU-compatible format to enable evaluation on systems without GPU support.

Table 6.1: Hyperparameter comparison of U-Net and FNO training configurations.

Parameter	**U-Net**	**FNO**
Optimizer	Adam	Adam + ClipNorm
Learning rate	$\{10^{-3}, 5\times10^{-3}, 10^{-4}, 5\times10^{-4}\}$	$\{10^{-3}, 5\times10^{-3}, 10^{-4}, 5\times10^{-4}\}$
Max. gradient norm	—	1.0
Loss	Composite (MSE + grad + bnd)	Composite (MSE + grad + bnd)
Batch size	8/16	8/16
Epochs (Exp. 1)	180	100
Input channels	4	4
Output channels	1	1
Base channels / width	32	$d_v = 64$
Architecture depth	4 encoder levels	$L = 4$ FNO blocks
Fourier modes $k_{\max}$	—	16
Checkpoint format	BSON	JLD2

7 Results

This chapter presents the results of the experiments described in Chapter 6. All experiments target the prediction of the stream function ψ for Stokes flow around settling crystals. The evaluation covers training convergence, quantitative error metrics on a held-out evaluation set, and qualitative inspection of predicted flow fields. Results are organized by experiment; within each experiment, FNO and U-Net are presented separately and then compared.

7.1 Experiment 1: Single-Crystal Baseline

Experiment 1 establishes a per-architecture baseline on the simplest configuration: single-crystal flows ($n = 1$). The sole free parameter across the four configurations is the learning rate; all other hyperparameters are fixed at the architecture defaults (see Section 6.5). The four configurations are referred to as *Exp-1.1* through *Exp-1.4*, with learning rates 10^{-3}, 5×10^{-3}, 10^{-4}, and 5×10^{-4}.

7.1.1 FNO – Training Behavior

Figure 7.1 shows the training and validation loss curves for all four FNO configurations over 100 epochs. All runs converge monotonically in training loss; the validation curves exhibit mild fluctuations but generally track the training loss, indicating no severe overfitting.

7.1.2 FNO – Evaluation Metrics

After training, each FNO model is evaluated on a fixed set of ten single-crystal samples from the held-out evaluation set. Table 7.1 summarizes the aggregated metrics. The divergence RMS is included as a physical sanity check; since the stream-function formulation guarantees $\nabla \cdot \mathbf{u} = 0$ by construction, its values are numerically negligible ($\sim 10^{-26}$) for all configurations.

For batch size 16, Exp-1.4 ($\mathrm{lr} = 5 \times 10^{-4}$) yields the lowest errors across all metrics after 100 epochs, achieving a mean relative L_2 error of 2.26% on ψ and 4.56% on the derived velocity field. Exp-1.2 (5×10^{-3}) and Exp-1.1 (10^{-3}) follow closely at 2.39% and 2.75% respectively, while Exp-1.3 (10^{-4}) remains the worst at 4.31%, as the lower learning rate

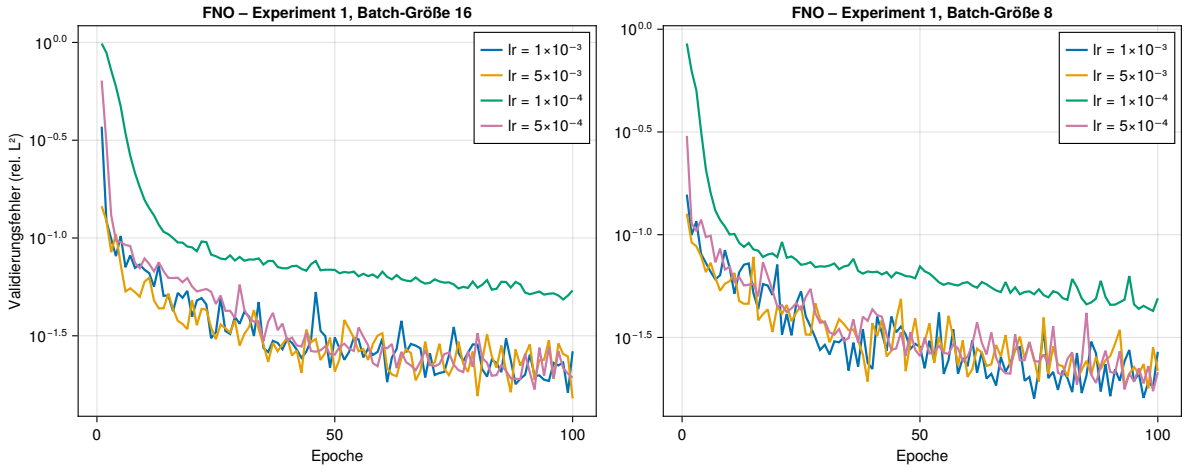

Figure 7.1: FNO Experiment 1: validation relative L_2 error over training epochs for all four learning-rate configurations, shown separately for batch size 16 (left) and batch size 8 (right). The y-axis uses a logarithmic scale.

Table 7.1: FNO evaluation metrics for Experiment 1 (single crystal, $n = 1$, 10 samples) for batch sizes 16 (upper block) and 8 (lower block). $\bar{\varepsilon}_\psi$: mean relative L_2 error on ψ; $\bar{\varepsilon}_v$: mean relative L_2 error on derived velocity; $\bar{e}_{\max}$: mean maximum absolute error on ψ; $\overline{\mathrm{MSE}}_\psi$: mean squared error on ψ. Standard deviations over the 10 samples in parentheses. Entries marked *(pending)* are not yet available.

Config (lr, bs)	$\bar{\varepsilon}_\psi$ (%)	$\bar{\varepsilon}_v$ (%)	$\bar{e}_{\max}$	$\overline{\mathrm{MSE}}_\psi$
Exp-1.1 (10^{-3}, bs 16)	2.75 ± 1.06	4.05 ± 1.00	0.538 ± 0.315	0.0306 ± 0.0309
Exp-1.2 (5×10^{-3}, bs 16)	2.39 ± 1.07	4.03 ± 0.67	0.503 ± 0.359	0.0261 ± 0.0297
Exp-1.3 (10^{-4}, bs 16)	4.31 ± 2.26	7.66 ± 1.32	0.815 ± 0.496	0.0878 ± 0.111
Exp-1.4 (5×10^{-4}, bs 16)	2.26 ± 0.87	4.56 ± 1.14	0.518 ± 0.207	0.0176 ± 0.0211
Exp-1.1 (10^{-3}, bs 8)	3.20 ± 1.46	3.91 ± 1.39	0.610 ± 0.392	0.0435 ± 0.0456
Exp-1.2 (5×10^{-3}, bs 8)	1.70 ± 0.77	3.07 ± 0.64	0.330 ± 0.190	0.0113 ± 0.0119
Exp-1.3 (10^{-4}, bs 8)	5.85 ± 1.99	10.56 ± 2.52	1.146 ± 0.472	0.1221 ± 0.0991
Exp-1.4 (5×10^{-4}, bs 8)	2.61 ± 0.85	3.82 ± 1.11	0.536 ± 0.371	0.0250 ± 0.0279

benefits less from the available training time. For batch size 8, Exp-1.2 (5×10^{-3}) achieves the best result overall with 1.70% / 3.07%, outperforming all bs 16 configurations across all metrics. Exp-1.4 (5×10^{-4}) follows at 2.61% / 3.82%, Exp-1.1 (10^{-3}) reaches 3.20% / 3.91%, and Exp-1.3 (10^{-4}) again lags behind at 5.85% / 10.56%, consistent with the pattern observed for bs 16.

Figure 7.3 (left panel) gives an overview of all bs 16 evaluation metrics across configurations.

7.1.3 FNO – Qualitative Results

Figure **??** shows representative predictions from Exp-1.4 (the best-performing FNO configuration for bs 16). The stream function field ψ and the derived velocity magnitude $|\mathbf{u}|$ are displayed alongside the reference solution.

7.1.4 U-Net – Training Behavior

Figure 7.2 shows the training and validation loss curves for the four U-Net configurations (batch size 16, 180 epochs each). The curves of Exp-1.1 (lr = 10^{-3}) and Exp-1.2 (5×10^{-3}) exhibit pronounced oscillations in validation loss, particularly in the early epochs, suggesting that the respective learning rates are too large for stable U-Net convergence without gradient clipping. Despite this, Exp-1.1 still reaches a best validation relative L_2 error of 3.5% at epoch 173. Exp-1.3 (10^{-4}) shows the smoothest convergence and achieves the lowest best-checkpoint validation error (3.0% at epoch 179). Exp-1.4 (5×10^{-4}) reaches 3.4% at epoch 163, closely following Exp-1.3. Exp-1.2 (5×10^{-3}) is the weakest configuration at 4.8%, likely due to the large learning rate causing it to overshoot the optimum early on.

7.1.5 U-Net – Metrics at Epoch 50

Table 7.2 reports the best-checkpoint validation metrics and the mean relative L_2 error on the held-out evaluation set (10 samples) for all eight U-Net configurations (four learning rates times two batch sizes).

For batch size 8, all four configurations cluster tightly in the range 5.5–6.5% on ψ, with Exp-1.2 (5×10^{-3}) achieving the best result (5.5% / 8.5%). The velocity errors remain roughly 1.5–2× larger than the ψ errors, which is a clear improvement over the earlier plain-MSE training, where the factor was approximately 3× (see Section 7.1.7). The gradient term in the composite loss explicitly penalizes errors in the derivatives of $\hat{\psi}$ and thereby suppresses high-frequency residuals that would otherwise be amplified by the finite-difference differentiation step.

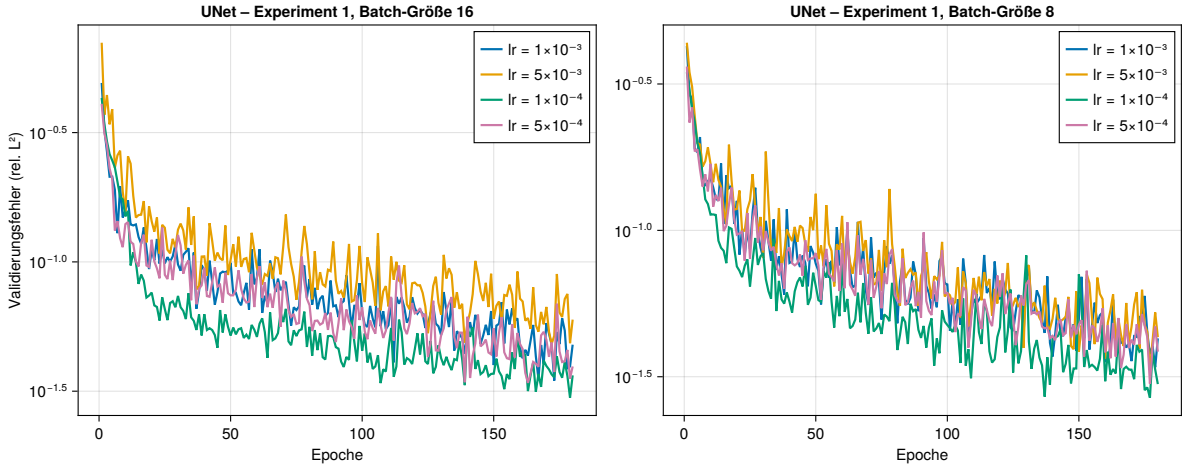

Figure 7.2: U-Net Experiment 1: validation relative L_2 error over training epochs for all four learning-rate configurations, shown separately for batch size 16 (left) and batch size 8 (right). The y-axis uses a logarithmic scale.

Table 7.2: U-Net metrics for Experiment 1 (single crystal, $n = 1$, 180 epochs, composite loss). $\bar{\varepsilon}_\psi^{\text{eval}}$: mean relative L_2 error on ψ, held-out evaluation set (10 samples, best checkpoint); $\bar{\varepsilon}_v^{\text{eval}}$: mean relative L_2 error on derived velocity; $\varepsilon_\psi^{\text{val}}$: best validation relative L_2 error on ψ over all 180 epochs.

Config (lr, bs)	$\bar{\varepsilon}_\psi^{\text{eval}}$ (%)	$\bar{\varepsilon}_v^{\text{eval}}$ (%)	$\varepsilon_\psi^{\text{val}}$ (%)
Exp-1.1 (10^{-3}, bs 16)	13.9	22.1	3.48
Exp-1.2 (5×10^{-3}, bs 16)	24.0	24.4	4.81
Exp-1.3 (10^{-4}, bs 16)	13.5	23.1	2.98
Exp-1.4 (5×10^{-4}, bs 16)	15.0	29.9	3.41
Exp-1.1 (10^{-3}, bs 8)	6.0	9.3	3.41
Exp-1.2 (5×10^{-3}, bs 8)	5.5	8.5	3.34
Exp-1.3 (10^{-4}, bs 8)	6.5	12.7	2.68
Exp-1.4 (5×10^{-4}, bs 8)	5.9	9.6	2.99

For batch size 16, the composite loss leads to substantially higher evaluation errors (13.5–24.0%) compared to the plain-MSE baseline (8.6–11.2%). This suggests that the additional loss terms introduce a training difficulty for the larger batch size, possibly due to noisier gradient estimates from the spatial derivative computation combined with the already less stable U-Net convergence at bs 16 (Section 7.1.4). Since bs 8 consistently outperforms bs 16 across all configurations, the remainder of this chapter focuses on bs 8 results for the U-Net.

7.1.6 U-Net – Qualitative Results

Figure **??** shows representative evaluation plots for Exp-1.2 (best evaluation-set performance for bs 8 with composite loss). The overall flow structure is reproduced qualitatively, but local errors near the crystal boundary appear more pronounced than in the corresponding FNO predictions. With composite loss training, velocity errors ($\approx 8.5\%$) are substantially reduced compared to the plain-MSE baseline ($\approx 16\%$), while ψ accuracy remains similar (5.5% vs. 5.2%).

7.1.7 U-Net – High-Frequency Residuals in the Velocity Field

7.1.8 Comparison: FNO vs. U-Net

Table 7.3 compares the best results from each architecture in Experiment 1 (FNO: 100 epochs; U-Net: 180 epochs, composite loss). For FNO, Exp-1.4 (bs 16) achieves a mean relative L_2 error of 2.26% on ψ and 4.56% on the derived velocity field; with bs 8 the best result is Exp-1.2 (5×10^{-3}) at 1.70% / 3.07%. The best U-Net result is Exp-1.2 (5×10^{-3}, bs 8) at 5.5% on ψ and 8.5% on velocity — a factor of roughly 3.2× above the FNO bs 16 optimum on ψ, but with a velocity-to-ψ error ratio of 1.5×, substantially closer to the FNO ratio ($\approx 2\times$) than the previous plain-MSE U-Net training ($\approx 3\times$).

Overall, the FNO converges to substantially lower error levels, suggesting a structural advantage of the spectral architecture for smooth, globally-structured Stokes flow fields around a single crystal. The adoption of the composite loss for the U-Net partially closes the gap in velocity accuracy, and confirms that the gradient term reduces high-frequency residuals regardless of architecture. Both models benefit from the stream-function output representation: the divergence RMS of all FNO predictions is on the order of 10^{-26}, confirming that incompressibility is preserved analytically irrespective of prediction accuracy.

Figure 7.3 gives a visual overview of all configurations for both metrics and both batch sizes.

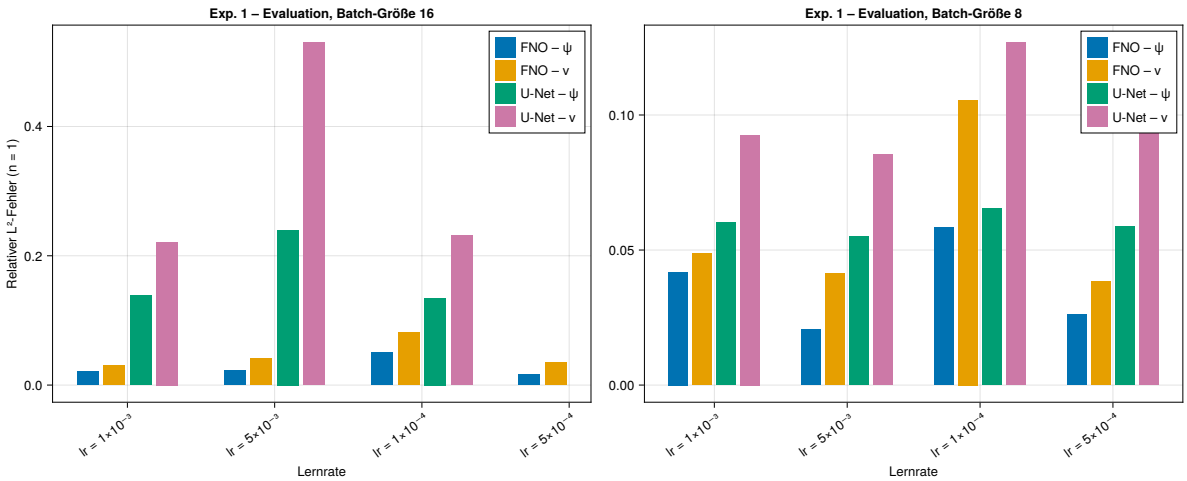

Figure 7.3: Experiment 1 evaluation metrics for all learning-rate configurations, shown separately for batch size 16 (left) and batch size 8 (right). Each group of bars shows the mean relative L_2 error on ψ and on the derived velocity field for FNO and U-Net side by side.

Table 7.3: Best result per architecture and batch size in Experiment 1 (single crystal, $n = 1$, FNO: 100 epochs; U-Net: 180 epochs, composite loss), evaluated on the held-out evaluation set (10 samples).

Architecture	**Config (lr, bs)**	$\bar{\varepsilon}_{\psi}^{\text{eval}}$ (%)	$\bar{\varepsilon}_{v}^{\text{eval}}$ (%)
FNO	Exp-1.4 (5×10^{-4}, bs 16)	2.26	4.56
FNO	Exp-1.2 (5×10^{-3}, bs 8)	1.70	3.07
U-Net	Exp-1.2 (5×10^{-3}, bs 8)	5.5	8.5
U-Net	Exp-1.3 (10^{-4}, bs 16)	13.5	23.1

7.2 Experiment 2: Multi-Crystal Generalization

Experiment 2 trains both models on samples with $n \in \{1, \ldots, 10\}$ crystals and evaluates each on all training crystal counts as well as on unseen higher counts up to $n = 25$. The U-Net is trained for 180 epochs; the FNO results presented here are from an earlier run at 50 epochs and will be updated once the extended run is complete.

Rather than fixing a single hyperparameter combination for all architectures, the configurations for Experiment 2 are derived directly from the Experiment 1 evaluation results: for each architecture and batch size, the learning rate that achieved the lowest held-out evaluation error in Experiment 1 is carried over.

For the **FNO**, this gives $\mathrm{lr} = 5\times10^{-4}$ for batch size 16 (Exp-1.4, 2.26%) and $\mathrm{lr} = 5\times10^{-3}$ for batch size 8 (Exp-1.2, 1.70%). The two batch-size variants represent different trade-offs: the larger batch with the lower learning rate offers more stable gradient estimates in the more complex multi-crystal loss landscape, while the smaller batch with the higher learning rate extrapolates the best-performing single-crystal setting directly.

For the **U-Net**, all batch-size 8 configurations cluster within a narrow range in Experiment 1 (5.5–6.5%), so the choice of learning rate is less critical. However, the configuration with the lowest validation error ($\mathrm{lr} = 10^{-4}$, Exp-1.3, 2.68% val) shows a notable gap to its evaluation performance (6.5% eval), which may indicate that the best checkpoint is overfit to the relatively small validation set rather than genuinely better. For this reason, $\mathrm{lr} = 10^{-3}$ is preferred for batch size 8 — it converges quickly, avoids this discrepancy, and is well-suited to the multi-crystal setting where the best checkpoint tends to appear early in training (Section 7.2.1). For batch size 16, $\mathrm{lr} = 10^{-4}$ achieved the best evaluation result in Experiment 1 (13.5%, Exp-1.3) and is retained here, though bs 16 consistently underperforms bs 8 across all U-Net configurations.

7.2.1 U-Net – Training Behavior

Figure **??** shows the training and validation loss curves over 180 epochs. The training MSE decreases monotonically, but the validation relative L_2 error fluctuates without reaching stable convergence; the best checkpoint is reached already at epoch 43 with a validation relative L_2 error of 28.5%. This suggests that the U-Net's best representable solution for the multi-crystal task is largely found in the early phase of training, with little further improvement thereafter.

7.2.2 U-Net – Evaluation Metrics

Table 7.4 summarizes the mean relative L_2 errors on ψ and the derived velocity field, stratified by crystal count and evaluated on 10 held-out samples per stratum. Crystal counts $n \in \{1, \ldots, 10\}$ are in-distribution; $n \in \{11, 12, 15, 20, 25\}$ are out-of-distribution

(OOD).

Table 7.4: U-Net evaluation metrics for Experiment 2 (trained on $n \in \{1, \ldots, 10\}$, 10 samples per stratum). OOD counts marked with †.

n		$\bar{\varepsilon}_\psi$ (%)	$\bar{\varepsilon}_v$ (%)
1		28.3 ± 5.2	66.9 ± 11.4
2		30.4 ± 11.5	73.1 ± 18.5
3		29.5 ± 18.6	78.4 ± 27.2
4		30.1 ± 8.6	79.7 ± 17.7
5		24.8 ± 15.3	78.8 ± 19.9
6		28.0 ± 14.2	73.2 ± 15.7
7		26.3 ± 11.8	73.0 ± 16.4
8		24.5 ± 9.3	66.6 ± 11.1
10		26.0 ± 15.6	70.2 ± 8.5
11	†	25.5 ± 7.5	69.6 ± 13.4
12	†	27.6 ± 6.1	68.6 ± 11.9
15	†	26.9 ± 13.0	82.1 ± 21.3
20	†	26.0 ± 7.8	92.9 ± 17.9
25	†	34.5 ± 7.4	94.7 ± 19.7

The in-distribution relative L_2 error on ψ clusters between 24% and 30% across all training crystal counts, a substantial improvement compared to the 50-epoch results (27–33%) and compared to the single-crystal baseline in Experiment 1 (8.6% for the best U-Net checkpoint at 180 epochs). The velocity errors are correspondingly high (67–79%), reflecting that the U-Net struggles to capture the complex superposition of wakes from multiple crystals.

Generalization to unseen crystal counts $n = 11$–12 is broadly consistent with in-distribution performance, suggesting that the model extrapolates smoothly to slightly higher crystal numbers. At $n = 20$–25, the velocity error rises to 93–95%, indicating a degradation in the accuracy of the derived flow field even though the ψ error remains comparable ($\sim$ 26–35%). Figure **??** shows the full evaluation metric profile stratified by crystal count.

7.2.3 U-Net – Qualitative Results

Figure **??** shows representative stream-function predictions for an in-distribution single-crystal sample and a seven-crystal sample, illustrating the qualitative prediction quality of the U-Net in the multi-crystal setting.

7.2.4 FNO – Training Behavior

Figure 7.4 shows the FNO training and validation loss curves over 50 epochs. Unlike the U-Net, both training MSE and validation relative L_2 error decrease steadily throughout training, indicating that the FNO converges well on the multi-crystal task with the chosen hyperparameters.

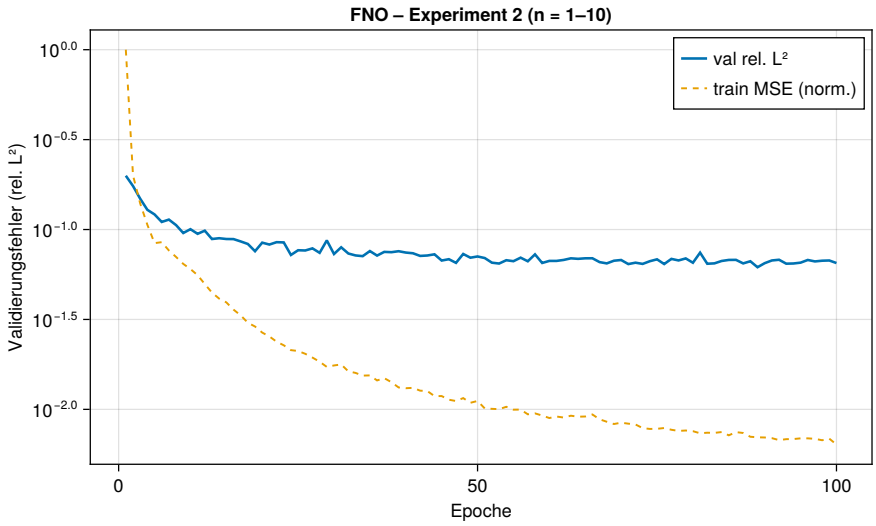

Figure 7.4: FNO Experiment 2: validation relative L_2 error and normalized training MSE over 50 training epochs. The validation error decreases steadily, in contrast to the oscillatory U-Net convergence in the same setting.

7.2.5 FNO – Evaluation Metrics

Table 7.5 reports the mean relative L_2 errors on ψ and the derived velocity field for all evaluated crystal counts.

Within the training distribution ($n \leq 10$), the FNO achieves ψ errors of 4–8% and velocity errors of 5–9%, substantially lower than the U-Net baseline (27–33%). The FNO generalizes well to $n = 11$–12 (errors only slightly above in-distribution) and remains reasonable at $n = 15$ (9.88%). However, at $n = 20$ the error increases sharply to 20.8%, and at $n = 25$ it reaches 31.8%, comparable to the U-Net's in-distribution error. This abrupt OOD degradation at $n \approx 20$ suggests that the FNO has learned representations tied to the density regime of the training distribution ($n \leq 10$) and struggles when that regime is exceeded.

Figure 7.5 (left panel) shows the full metric profile stratified by crystal count.

7.2.6 FNO – Qualitative Results

Figure **??** illustrates FNO predictions for an in-distribution seven-crystal sample and an OOD twenty-five-crystal sample, highlighting the divergence in prediction quality between

Table 7.5: FNO evaluation metrics for Experiment 2 (trained on $n \in \{1, \ldots, 10\}$, 10 samples per stratum). OOD counts marked with †.

n		$\bar{\varepsilon}_\psi$ (%)	$\bar{\varepsilon}_v$ (%)
1		3.98 ± 1.29	5.01 ± 1.23
2		5.45 ± 3.04	6.06 ± 3.00
3		4.37 ± 1.81	5.33 ± 1.92
4		6.38 ± 3.35	6.81 ± 2.95
5		5.25 ± 2.47	6.00 ± 2.02
6		5.60 ± 3.76	6.54 ± 3.31
7		7.98 ± 4.97	8.74 ± 4.63
8		5.66 ± 4.96	6.80 ± 4.42
10		8.38 ± 5.57	8.65 ± 3.58
11	†	6.94 ± 3.26	7.96 ± 3.29
12	†	9.17 ± 3.95	9.88 ± 3.29
15	†	9.88 ± 5.70	11.6 ± 5.61
20	†	20.8 ± 2.62	22.6 ± 3.52
25	†	31.8 ± 6.88	32.5 ± 7.24

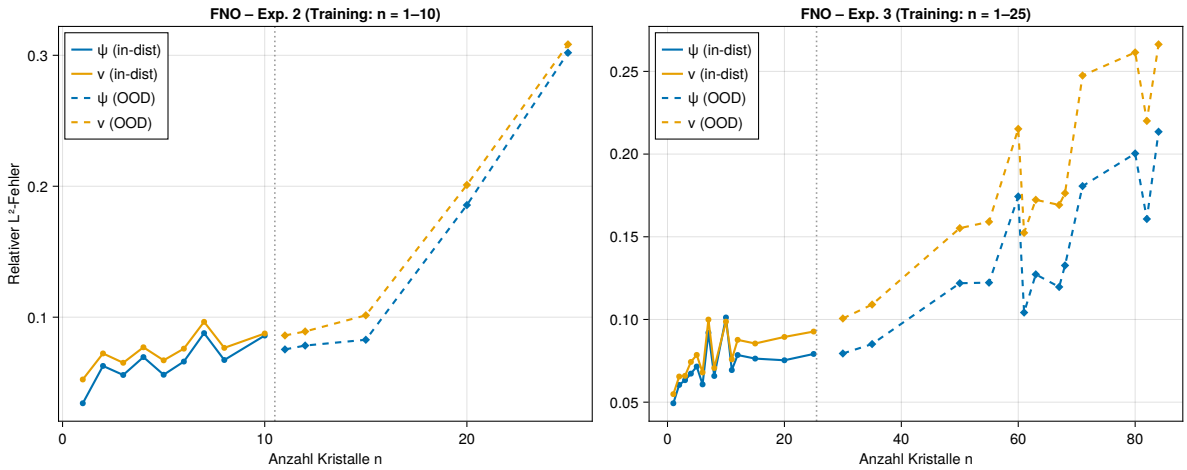

Figure 7.5: FNO generalization across crystal counts for Experiment 2 (left, trained on $n \leq 10$) and Experiment 3 (right, trained on $n \leq 25$). Solid lines indicate in-distribution counts; dashed lines with diamond markers indicate out-of-distribution counts. The dotted vertical line marks the boundary of the training distribution.

these regimes.

7.2.7 Comparison: FNO vs. U-Net

Table 7.6 compares the mean in-distribution ψ error and the OOD error at $n = 25$ for both architectures.

Table 7.6: FNO vs. U-Net comparison for Experiment 2 (trained on $n \in \{1, \ldots, 10\}$). In-distribution: mean over $n = 1$–10; OOD: at $n = 25$.

Architecture	$\bar{\varepsilon}_{\psi}^{\text{in-dist}}$ (%)	$\bar{\varepsilon}_{v}^{\text{in-dist}}$ (%)	$\bar{\varepsilon}_{\psi}^{\text{OOD},n=25}$ (%)
FNO	~ 6.1	~ 6.8	31.8
U-Net	~ 27	~ 73	34.5

The FNO outperforms the U-Net by a factor of roughly 5× within the training distribution. For extreme OOD counts ($n = 25$, 2.5× beyond the training maximum), both architectures degrade to similar error levels, underscoring that OOD generalization is a challenge independent of architecture for this level of distribution shift.

7.3 Experiment 3: Stress-Testing Generalization Limits

Experiment 3 trains both models on configurations with up to $n = 25$ crystals and evaluates each on substantially more complex scenes with up to $n = 84$ crystals to probe the limits of out-of-distribution generalization. The U-Net is trained for 180 epochs; the FNO results presented here are from an earlier run at 50 epochs and will be updated once the extended run is complete.

The hyperparameter configurations follow the same rationale as Experiment 2: the learning rates identified as best-performing for each architecture and batch size in Experiment 1 are used without modification. For the FNO, this means $\text{lr} = 5 \times 10^{-4}$ with batch size 16 and $\text{lr} = 5 \times 10^{-3}$ with batch size 8. For the U-Net, $\text{lr} = 10^{-3}$ is used with batch size 8 and $\text{lr} = 10^{-4}$ with batch size 16. The extended training distribution ($n \leq 25$ instead of $n \leq 10$) increases the complexity of each training batch but does not change the reasoning behind the choice of learning rate and batch size.

7.3.1 U-Net – Training Behavior

Figure **??** shows the training and validation loss curves over 180 epochs. The behavior is qualitatively similar to Experiment 2: training MSE decreases steadily while validation relative L_2 error oscillates without reaching stable convergence. The best checkpoint is reached at epoch 49 with a validation relative L_2 error of 28.6%, comparable to Experi-

ment 2 (28.5% at epoch 43), suggesting that the wider training distribution provides no significant benefit for U-Net convergence behavior.

7.3.2 U-Net – Evaluation Metrics

Table 7.7 reports the mean relative L_2 errors on ψ and the derived velocity, evaluated on 10 held-out samples per stratum. Crystal counts $n \in \{1, \ldots, 25\}$ are in-distribution; $n \geq 26$ are OOD. For OOD strata with fewer than 10 available samples, the table reports the number of samples actually used.

Table 7.7: U-Net evaluation metrics for Experiment 3 (trained on $n \in \{1, \ldots, 25\}$). OOD counts ($n > 25$) marked with †; N denotes actual sample count.

n	N		$\bar{\varepsilon}_\psi$ (%)	$\bar{\varepsilon}_v$ (%)
1	10		36.7 ± 7.8	90.3 ± 23.7
2	10		32.6 ± 9.3	81.8 ± 16.5
3	10		28.4 ± 12.7	86.9 ± 20.7
5	10		24.4 ± 12.2	88.6 ± 22.1
7	10		25.1 ± 8.5	78.0 ± 16.3
10	10		26.2 ± 13.6	74.7 ± 11.6
15	10		26.4 ± 11.4	86.3 ± 28.0
20	10		23.7 ± 7.3	92.1 ± 20.5
25	10		28.3 ± 5.8	92.2 ± 20.2
30	10	†	29.5 ± 9.7	105.5 ± 18.8
35	10	†	29.5 ± 10.2	106.9 ± 20.0
50	10	†	28.1 ± 6.1	105.2 ± 13.5
55	10	†	33.5 ± 7.4	122.1 ± 11.8
67	1	†	39.0	128.5
68	2	†	53.3 ± 20.8	121.3 ± 1.0
71	1	†	26.9	122.1
80	1	†	34.2	113.6
82	1	†	70.1	126.9
84	1	†	63.4	100.3

Within the training distribution ($n \leq 25$), the ψ errors range from 24% to 37%, broadly comparable to Experiment 2. Velocity errors are noticeably high for both small and large in-distribution counts (up to 92% at $n = 25$), indicating that the U-Net struggles with the superposition of flow contributions even within the training range.

For OOD crystal counts $n = 30$–55, the model degrades gradually: ψ errors remain in the 28–34% range, while velocity errors exceed 100% throughout. At $n = 55$, the velocity error rises to 122%. For very high counts ($n = 67$–84), individual ψ errors vary widely from 27% to 70%, reflecting high sensitivity to the specific crystal arrangement at such extreme densities. The anomalously high errors at $n = 82$ and $n = 84$ suggest that the flow field is qualitatively misrepresented for those particular configurations.

Figure ?? shows the full metric profile.

7.3.3 U-Net – Qualitative Results

Figure ?? illustrates stream-function predictions for an in-distribution seven-crystal sample and an OOD fifty-crystal sample.

7.3.4 FNO – Training Behavior

Figure 7.6 shows the FNO training and validation loss curves over 50 epochs. As in Experiment 2, training converges steadily, but the final validation relative L_2 error is somewhat higher than in the single-crystal baseline, reflecting the increased task complexity.

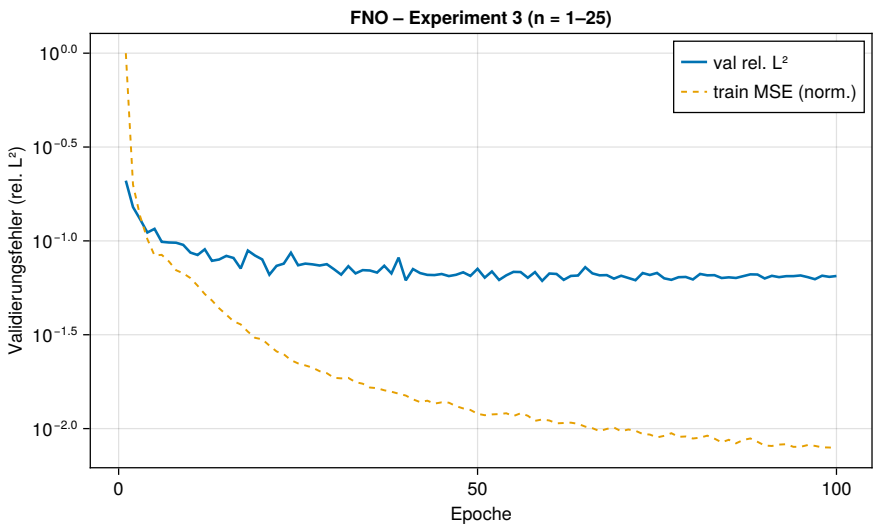

Figure 7.6: FNO Experiment 3: validation relative L_2 error and normalized training MSE over 50 training epochs (training distribution: $n \leq 25$). Convergence is steady throughout, with a slightly higher final error than in Experiment 2.

7.3.5 FNO – Evaluation Metrics

Table 7.8 reports the mean relative L_2 errors on ψ and the derived velocity field. Crystal counts $n \leq 25$ are in-distribution; $n \geq 30$ are OOD.

Within the training distribution ($n \leq 25$), the FNO achieves ψ errors of 4–8%, consistent with the Experiment 2 in-distribution performance. The OOD behavior is markedly improved compared to Experiment 2: at $n = 30$ (just beyond the training boundary) the error rises only slightly to 6.5%, and at $n = 35$ to 8.2%. A more pronounced increase occurs around $n = 50$, where the error reaches 14.2%, and beyond $n = 55$ the errors grow further to 20–25% at the highest evaluated counts ($n = 80, 84$). This gradual degradation

Table 7.8: FNO evaluation metrics for Experiment 3 (trained on $n \in \{1, \ldots, 25\}$). OOD counts ($n > 25$) marked with †; N denotes actual sample count.

n	N		$\bar{\varepsilon}_\psi$ (%)	$\bar{\varepsilon}_v$ (%)
1	10		3.75 ± 0.91	4.59 ± 1.08
3	10		5.50 ± 2.81	6.09 ± 2.67
5	10		4.89 ± 3.12	5.85 ± 2.96
7	10		6.61 ± 4.44	7.42 ± 4.24
10	10		7.21 ± 5.37	7.67 ± 3.67
15	10		6.54 ± 4.49	7.57 ± 3.43
20	10		4.66 ± 1.60	6.16 ± 1.51
25	10		7.58 ± 4.09	8.61 ± 3.84
30	10	†	6.48 ± 2.66	8.33 ± 2.32
35	10	†	8.23 ± 4.51	10.1 ± 3.88
50	10	†	14.2 ± 4.66	16.1 ± 4.26
55	10	†	12.7 ± 3.90	15.1 ± 3.75
60	1	†	16.1	18.5
71	1	†	20.5	25.0
80	1	†	23.9	28.1
84	1	†	25.4	28.2

contrasts sharply with Experiment 2's abrupt failure at $n = 20$, demonstrating that training on a wider crystal-count distribution substantially extends the FNO's generalization range. The right panel of Figure 7.5 shows the full metric profile.

7.3.6 FNO – Qualitative Results

Figure **??** illustrates FNO predictions for an in-distribution seven-crystal sample and an OOD fifty-crystal sample.

7.3.7 Comparison: FNO vs. U-Net

Table 7.9 compares FNO and U-Net at representative in-distribution and OOD crystal counts.

Table 7.9: FNO vs. U-Net comparison for Experiment 3 (trained on $n \in \{1, \ldots, 25\}$). In-distribution: mean over $n = 1, 5, 10, 20, 25$; OOD: at $n = 50$.

Architecture	$\bar{\varepsilon}_\psi^{\text{in-dist}}$ (%)	$\bar{\varepsilon}_v^{\text{in-dist}}$ (%)	$\bar{\varepsilon}_\psi^{\text{OOD},n=50}$ (%)
FNO	~ 5.7	~ 6.6	14.2
U-Net	~ 28	~ 85	28.1

Across both experiments, the FNO consistently outperforms the U-Net within the training distribution by a factor of roughly 5×. In the OOD regime the advantage narrows:

at $n = 50$, the FNO's 14.2% error and the U-Net's 28.1% differ by a factor of 2× — a smaller gap than in the 50-epoch run, where the U-Net reached 31.4% at $n = 50$. The velocity errors tell a similar story: the FNO's ∼16% at $n = 50$ contrasts with the U-Net's ∼105%, underscoring the spectral architecture's superior ability to capture global flow structure even for unseen crystal configurations.

7.4 Experiment 4: Architecture Ablation

[TODO: Ergebnisse werden nach Abschluss der Experimente ergänzt (vgl. Abschnitt 6.4).]

8 Discussion

Hier soll eine Diskussion der Ergebnisse meiner Experimente stattfinden. Ich werde die Leistung meines Modells anhand der Metriken bewerten, die ich in Kapitel 6 beschrieben habe, und die Vorhersagen meines Modells mit den Referenzlösungen vergleichen. Es wird auch eine Diskussion darüber geben, welche Aspekte meiner Modellierung gut funktionieren und wo es noch Herausforderungen gibt. Ich werde die Ergebnisse in Bezug auf die verschiedenen Lernziele und Eingabekodierungen analysieren und versuchen, Muster oder Trends zu identifizieren, die auf die Stärken und Schwächen meines Modells hinweisen. Schließlich werde ich die Ergebnisse in den Kontext der bestehenden Literatur einordnen und mögliche Implikationen für zukünftige Arbeiten diskutieren. Wobei ich sicherstellen muss, dass ich hier nicht einfach die Ergebnisse aus Kapitel 8 wiederhole, sondern wirklich eine tiefere Analyse und Interpretation der Ergebnisse durchführe. Es soll nicht nur darum gehen, was die Ergebnisse sind, sondern auch warum sie so sind und was sie für die Praxis und die Forschung bedeuten. Es könnte auch eine Diskussion darüber geben, welche Einschränkungen oder Unsicherheiten es bei meinen Ergebnissen gibt und wie diese berücksichtigt werden sollten. Insgesamt soll dieses Kapitel dazu beitragen, die Bedeutung meiner Ergebnisse zu verstehen und ihre Auswirkungen auf die Praxis und die Forschung zu diskutieren.

A Bibliography

[1] Ashwin, Neil Raj et al. (Apr. 2024). "Physics informed deep learning for flow and force predictions in dense ellipsoidal particle suspensions". en. In: *Powder Technology* 439, p. 119684. ISSN: 00325910. DOI: 10.1016/j.powtec.2024.119684. URL: https://linkinghub.elsevier.com/retrieve/pii/S0032591024003267 (visited on 02/15/2026).

[2] Chen, Junfeng, Jonathan Viquerat, and Elie Hachem (2019). "U-net architectures for fast prediction of incompressible laminar flows". en. In.

[3] Guo, Xiaoxiao, Wei Li, and Francesco Iorio (Aug. 2016). "Convolutional Neural Networks for Steady Flow Approximation". en. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. San Francisco California USA: ACM, pp. 481–490. ISBN: 978-1-4503-4232-2. DOI: 10.1145/2939672.2939738. URL: https://dl.acm.org/doi/10.1145/2939672.2939738 (visited on 02/15/2026).

[4] Hu, Beichao and Dwayne McDaniel (Oct. 2023). "Applying Physics-Informed Neural Networks to Solve Navier–Stokes Equations for Laminar Flow around a Particle". en. In: *Mathematical and Computational Applications* 28.5, p. 102. ISSN: 2297-8747. DOI: 10.3390/mca28050102. URL: https://www.mdpi.com/2297-8747/28/5/102 (visited on 02/15/2026).

[5] Jin, Xiaowei et al. (Feb. 2021). "NSFnets (Navier-Stokes flow nets): Physics-informed neural networks for the incompressible Navier-Stokes equations". en. In: *Journal of Computational Physics* 426, p. 109951. ISSN: 00219991. DOI: 10.1016/j.jcp.2020.109951. URL: https://linkinghub.elsevier.com/retrieve/pii/S0021999120307257 (visited on 02/15/2026).

[6] Kovachki, Nikola et al. (2021). "Neural Operator: Learning Maps Between Function Spaces With Applications to PDEs". en. In.

[7] Ladd, Anthony J C (1994). "Numerical Simulations of Particulate Suspensions via a Discretized Boltzmann Equation Part I. Theoretical Foundation". en. In.

[8] Le, Tuyen Quang, Pao-Hsiung Chiu, and Chinchun Ooi (Sept. 2022). "U-Net-Based Surrogate Model for Evaluation of Microfluidic Channels". en. In: *International Journal of Computational Methods* 19.07, p. 2141018. ISSN: 0219-8762, 1793-6969. DOI: 10.1142/S0219876221410188. URL: https://www.worldscientific.com/doi/10.1142/S0219876221410188 (visited on 02/15/2026).

[9] Leonardi, Alessandro et al. (May 2014). "Coupled DEM-LBM method for the free-surface simulation of heterogeneous suspensions". In: *Comp. Part. Mech.* 1.1, pp. 3–13. ISSN: 2196-4378, 2196-4386. DOI: 10.1007/s40571-014-0001-z. URL: http://link.springer.com/10.1007/s40571-014-0001-z (visited on 11/24/2025).

[10] Li, Xiaohui et al. (2022). "IBM-LBM-DEM Study of Two-Particle Sedimentation: Drafting-Kissing-Tumbling and Effects of Particle Reynolds Number and Initial Positions of Particles". In: *Energies* 15.9. ISSN: 1996-1073. DOI: 10.3390/en15093297. URL: https://www.mdpi.com/1996-1073/15/9/3297.

[11] Li, Zongyi et al. (May 2021). *Fourier Neural Operator for Parametric Partial Differential Equations*. en. arXiv:2010.08895 [cs]. DOI: 10.48550/arXiv.2010.08895. URL: http://arxiv.org/abs/2010.08895 (visited on 02/15/2026).

[12] Lu, Lu et al. (Apr. 2022). "A comprehensive and fair comparison of two neural operators (with practical extensions) based on FAIR data". en. In: *Computer Methods in Applied Mechanics and Engineering* 393, p. 114778. ISSN: 00457825. DOI: 10.1016/j.cma.2022.114778. URL: https://linkinghub.elsevier.com/retrieve/pii/S0045782522001207 (visited on 02/15/2026).

[13] Ma, Zhan, Zisheng Ye, and Wenxiao Pan (Oct. 2022). "Fast simulation of particulate suspensions enabled by graph neural network". en. In: *Computer Methods in Applied Mechanics and Engineering* 400, p. 115496. ISSN: 00457825. DOI: 10.1016/j.cma.2022.115496. URL: https://linkinghub.elsevier.com/retrieve/pii/S0045782522005114 (visited on 02/15/2026).

[14] Martin, D. and R. Nokes (Dec. 1989). "A Fluid-Dynamical Study of Crystal Settling in Convecting Magmas". en. In: *Journal of Petrology* 30.6, pp. 1471–1500. ISSN: 0022-3530, 1460-2415. DOI: 10.1093/petrology/30.6.1471. URL: https://academic.oup.com/petrology/article-lookup/doi/10.1093/petrology/30.6.1471 (visited on 02/15/2026).

[15] Martin, Daniel and Roger Nokes (Apr. 1988). "Crystal settling in a vigorously convecting magma chamber". en. In: *Nature* 332.6164, pp. 534–536. ISSN: 0028-0836, 1476-4687. DOI: 10.1038/332534a0. URL: https://www.nature.com/articles/332534a0 (visited on 02/15/2026).

[16] Morimoto, Masaki et al. (Mar. 2022). "Generalization techniques of neural networks for fluid flow estimation". en. In: *Neural Computing and Applications* 34.5, pp. 3647–3669. ISSN: 0941-0643, 1433-3058. DOI: 10.1007/s00521-021-06633-z. URL: https://link.springer.com/10.1007/s00521-021-06633-z (visited on 02/15/2026).

[17] Nissanka, Kavinda J., Xiaolei Ma, and Justin C. Burton (Feb. 2023). "Dynamics of Mass Polar Spheroids During Sedimentation". en. In: *Journal of Fluid Mechanics* 956. arXiv:2206.09505 [cond-mat], A28. ISSN: 0022-1120, 1469-7645. DOI: 10.1017/jfm.2023.32. URL: http://arxiv.org/abs/2206.09505 (visited on 02/15/2026).

[18] Oldenburg, Jan et al. (June 2022). "Geometry aware physics informed neural network surrogate for solving Navier–Stokes equation (GAPINN)". en. In: *Advanced Modeling and Simulation in Engineering Sciences* 9.1, p. 8. ISSN: 2213-7467. DOI: 10.1186/s40323-022-00221-z. URL: https://amses-journal.springeropen.com/articles/10.1186/s40323-022-00221-z (visited on 02/15/2026).

[19] Patocka, Vojtech, Enrico Calzavarini, and Nicola Tosi (Nov. 2020). "Settling of inertial particles in turbulent Rayleigh-Benard convection". en. In: *Physical Review Fluids* 5.11. arXiv:2005.05448 [physics], p. 114304. ISSN: 2469-990X. DOI: 10.1103/PhysRevFluids.5.114304. URL: http://arxiv.org/abs/2005.05448 (visited on 02/15/2026).

[20] Penlou, Baptiste et al. (Mar. 2023). "Experimental Measurement of Enhanced and Hindered Particle Settling in Turbulent Gas-Particle Suspensions, and Geophysical Implications". en. In: *Journal of Geophysical Research: Solid Earth* 128.3, e2022JB025809. ISSN: 2169-9313, 2169-9356. DOI: 10.1029/2022JB025809. URL: https://agupubs.onlinelibrary.wiley.com/doi/10.1029/2022JB025809 (visited on 02/15/2026).

[21] Popov, Anton and Boris J. P. Kaus (2013). "LaMEM (Lithosphere and Mantle Evolution Model): advancing a staggered-grid finite difference version of the code". In: *EGU General Assembly 2013*. Vol. 15. Source code available at https://github.com/UniMainzGeo/LaMEM, EGU2013–7761.

[22] Qin, Shaoxiang et al. (Oct. 2024). *Toward a Better Understanding of Fourier Neural Operators from a Spectral Perspective*. en. arXiv:2404.07200 [cs]. DOI: 10.48550/arXiv.2404.07200. URL: http://arxiv.org/abs/2404.07200 (visited on 02/15/2026).

[23] Raissi, M., P. Perdikaris, and G. E. Karniadakis (2019). "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations". In: *Journal of Computational Physics* 378, pp. 686–707. ISSN: 0021-9991. DOI: https://doi.org/10.1016/j.jcp.2018.10.045. URL: https://www.sciencedirect.com/science/article/pii/S0021999118307125.

[24] Raissi, Maziar, Paris Perdikaris, and George Em Karniadakis (Nov. 2017). *Physics Informed Deep Learning (Part I): Data-driven Solutions of Nonlinear Partial Differential Equations*. en. arXiv:1711.10561 [cs]. DOI: 10.48550/arXiv.1711.10561. URL: http://arxiv.org/abs/1711.10561 (visited on 02/15/2026).

[25] Rana, Pratip et al. (Oct. 2024). "A scalable convolutional neural network approach to fluid flow prediction in complex environments". en. In: *Scientific Reports* 14.1, p. 23080. ISSN: 2045-2322. DOI: 10.1038/s41598-024-73529-y. URL: https://www.nature.com/articles/s41598-024-73529-y (visited on 02/15/2026).

[26] Raonić, Bogdan et al. (Dec. 2023). *Convolutional Neural Operators for robust and accurate learning of PDEs*. en. arXiv:2302.01178 [cs]. DOI: 10.48550/arXiv.2302.01178. URL: http://arxiv.org/abs/2302.01178 (visited on 02/15/2026).

[27] Ribeiro, Mateus Dias et al. (2021). "DeepCFD: Efficient Steady-State Laminar Flow Approximation with Deep Convolutional Neural Networks". en. In.

[28] Richter-Powell, Jack, Yaron Lipman, and Ricky T Q Chen (n.d.). "Neural Conservation Laws: A Divergence-Free Perspective". en. In: ().

[29] Ronneberger, Olaf, Philipp Fischer, and Thomas Brox (May 2015). *U-Net: Convolutional Networks for Biomedical Image Segmentation*. en. arXiv:1505.04597 [cs]. DOI: 10.48550/arXiv.1505.04597. URL: http://arxiv.org/abs/1505.04597 (visited on 02/15/2026).

[30] Sanchez-Gonzalez, Alvaro et al. (2020). "Learning to Simulate Complex Physics with Graph Networks". en. In.

[31] Sun, Luning et al. (Apr. 2020). "Surrogate Modeling for Fluid Flows Based on Physics-Constrained Deep Learning Without Simulation Data". en. In: *Computer Methods in Applied Mechanics and Engineering* 361. arXiv:1906.02382 [physics], p. 112732. ISSN: 00457825. DOI: 10.1016/j.cma.2019.112732. URL: http://arxiv.org/abs/1906.02382 (visited on 02/15/2026).

[32] Takamoto, Makoto et al. (2022). "PDEBENCH: An Extensive Benchmark for Scientific Machine Learning". en. In.

[33] Thieulot, Cedric and Wolfgang Bangerth (Jan. 2022). "On the choice of finite element for applications in geodynamics". en. In: *Solid Earth* 13.1, pp. 229–249. ISSN: 1869-9529. DOI: 10.5194/se-13-229-2022. URL: https://se.copernicus.org/articles/13/229/2022/ (visited on 02/15/2026).

[34] Thuerey, Nils et al. (Jan. 2020). "Deep Learning Methods for Reynolds-Averaged Navier–Stokes Simulations of Airfoil Flows". en. In: *AIAA Journal* 58.1, pp. 25–36. ISSN: 0001-1452, 1533-385X. DOI: 10.2514/1.J058291. URL: https://arc.aiaa.org/doi/10.2514/1.J058291 (visited on 02/15/2026).

[35] Uhlmann, Markus and Todor Doychev (Aug. 2014). "Sedimentation of a dilute suspension of rigid spheres at intermediate Galileo numbers: the effect of clustering upon the particle motion". en. In: *Journal of Fluid Mechanics* 752. arXiv:1406.1667 [physics], pp. 310–348. ISSN: 0022-1120, 1469-7645. DOI: 10.1017/jfm.2014.330. URL: http://arxiv.org/abs/1406.1667 (visited on 02/15/2026).

[36] Verhoeven, J. and J. Schmalzl (Dec. 2009). "A numerical method for investigating crystal settling in convecting magma chambers". en. In: *Geochemistry, Geophysics, Geosystems* 10.12, 2009GC002509. ISSN: 1525-2027, 1525-2027. DOI: 10.1029/2009GC002509. URL: https://agupubs.onlinelibrary.wiley.com/doi/10.1029/2009GC002509 (visited on 02/15/2026).

[37] Weinstein, Stuart A., David A. Yuen, and Peter L. Olson (1988). "Evolution of crystal-settling in magma-chamber convection". In: *Earth and Planetary Science Letters* 87.1, pp. 237–248. ISSN: 0012-821X. DOI: https://doi.org/10.1016/0012-821X(88)90078-7. URL: https://www.sciencedirect.com/science/article/pii/0012821X88900787.

[38] Wen, Gege et al. (May 2022). "U-FNO—An enhanced Fourier neural operator-based deep-learning model for multiphase flow". en. In: *Advances in Water Resources* 163, p. 104180. ISSN: 03091708. DOI: 10.1016/j.advwatres.2022.104180. URL: https://linkinghub.elsevier.com/retrieve/pii/S0309170822000562 (visited on 02/15/2026).

[39] Zhu, Yinhao and Nicholas Zabaras (Aug. 2018). "Bayesian Deep Convolutional Encoder-Decoder Networks for Surrogate Modeling and Uncertainty Quantification". en. In: *Journal of Computational Physics* 366. arXiv:1801.06879 [physics], pp. 415–447. ISSN: 00219991. DOI: 10.1016/j.jcp.2018.04.018. URL: http://arxiv.org/abs/1801.06879 (visited on 02/15/2026).

[40] Zhu, Yinhao, Nicholas Zabaras, et al. (Oct. 2019). "Physics-Constrained Deep Learning for High-dimensional Surrogate Modeling and Uncertainty Quantification without Labeled Data". en. In: *Journal of Computational Physics* 394. arXiv:1901.06314 [physics], pp. 56–81. ISSN: 00219991. DOI: 10.1016/j.jcp.2019.05.024. URL: http://arxiv.org/abs/1901.06314 (visited on 02/15/2026).